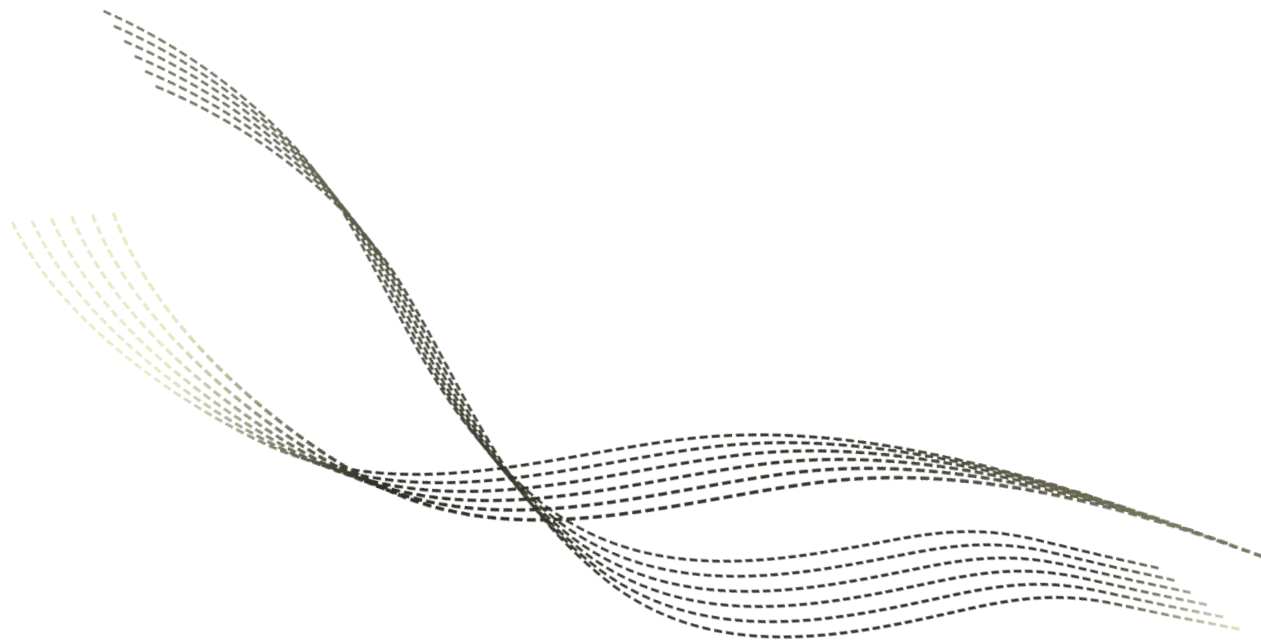


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路布线方法

沈明华



目录

CONTENTS

01

布线

02

通用布线算法

03

全局布线算法

04

详细布线算法

PART 03

全局布线算法

■ 全局布线算法

□ 问题定义

– 输入

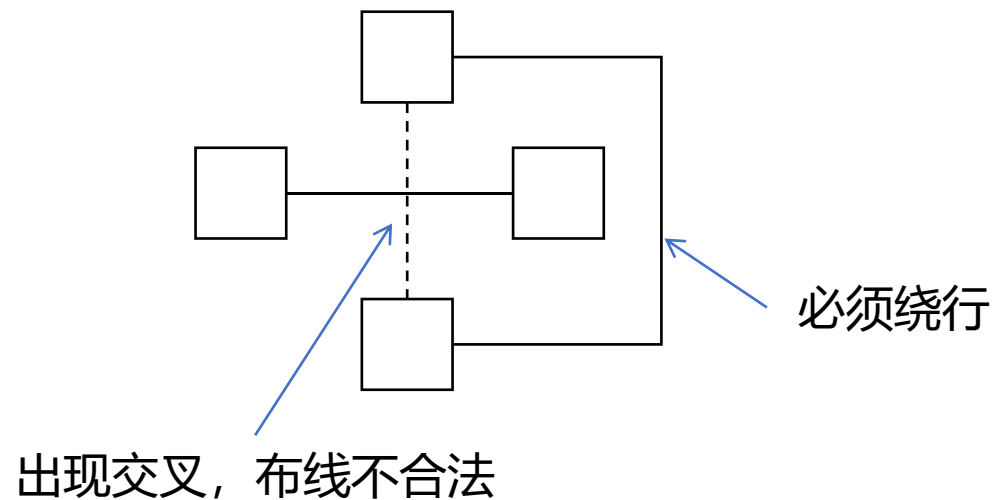
- 给定线网(net)集合 $e \in V$ 和上一阶段的布局结果 S

– 输出

- 一种合法的布线结果 $\mathcal{L}$, 给出每条线网 $e \in V$ 具体的走线细节
- 所有线网的总长度最短

– 约束

- 线网之间不能出现交叉
- 所有模块按照要求连接



■ 全局布线算法

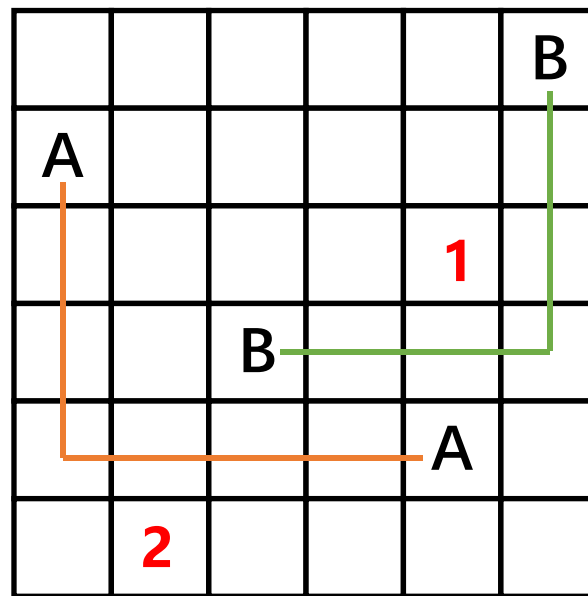
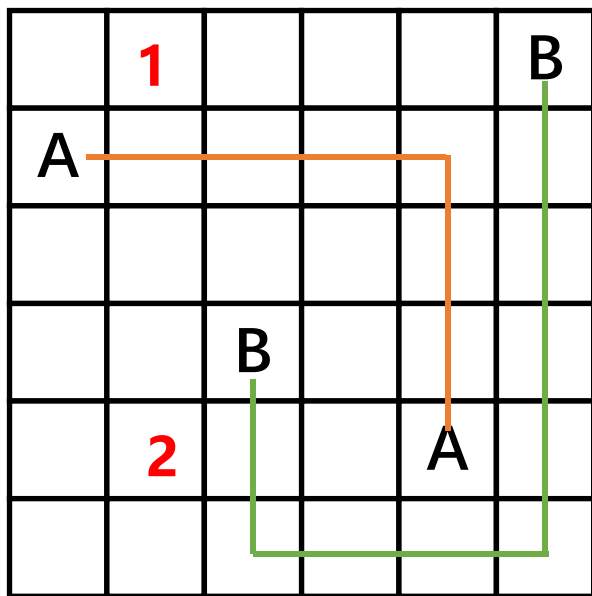
□ 顺序全局布线(Sequential global routing)

- 选择一个特定的线网(net)顺序，然后按该顺序依次进行布线
 - 布线使用上一章提到的通用布线算法
- 然而，这种顺序布线方法通常会导致较差的布线结果，因为先布线的线网可能会阻碍后续线网的布线
- 布线解决方案的质量在很大程度上取决于线网的顺序

■ 全局布线算法

□ 顺序全局布线

- 对于以下两个双引脚线网的例子
- 如果先布线A再布线B，可能得到的一个较差的布线结果
- 如果先布线B再布线A，容易得到的一个较好的布线结果



■ 全局布线算法

□ 顺序全局布线

- 在早期的一项研究中，Abel得出结论：不存在一种单一的线网排序方案能够在所有布线问题中优于其他排序方案^[1]。
 - 找到最优的线网顺序已被证明是一个NP难的问题，很可能不存在多项式时间算法来解决这个问题
- 现在线网排序通常采用启发式策略
 - 例如，按照线网覆盖范围内其他线网的引脚数量从少到多
 - 如果一个线网的覆盖范围内有更多其他线网的引脚，那么它更容易阻碍此区域内的其他线网的布线
- 可以通过拆线与重新布线(Rip-up and Reroute)进一步优化布线结果

[1] L. C. Abel, "On the ordering of connections for automatic wire routing" , TC, 1972.

■ 全局布线算法

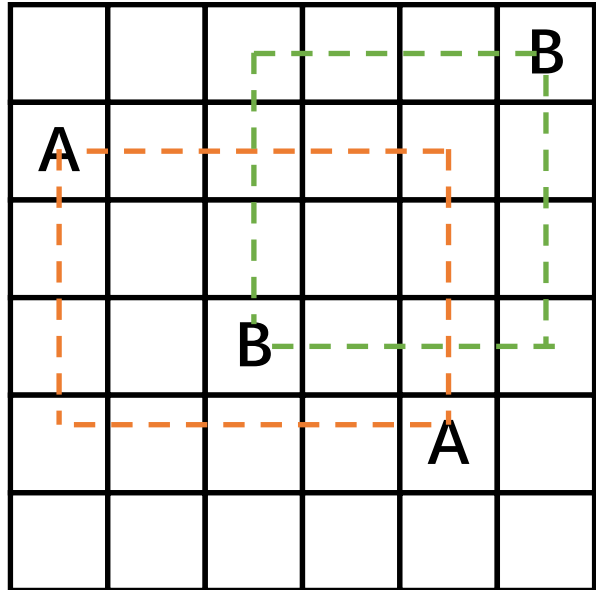
□ 顺序全局布线

– 启发式的线网排序

- 先按照线网边界框内引脚数量，从少到多排序线网
- 按排序的线网顺序使用通用布线算法依次布线

– 如右图所示，橙色虚线即为A的边界框，绿色虚线即为B的边界框

- A覆盖范围内其他线网的引脚数量为1，包含1个B线网引脚
- B覆盖范围内其他线网的引脚数量为0
- 因此B先布线

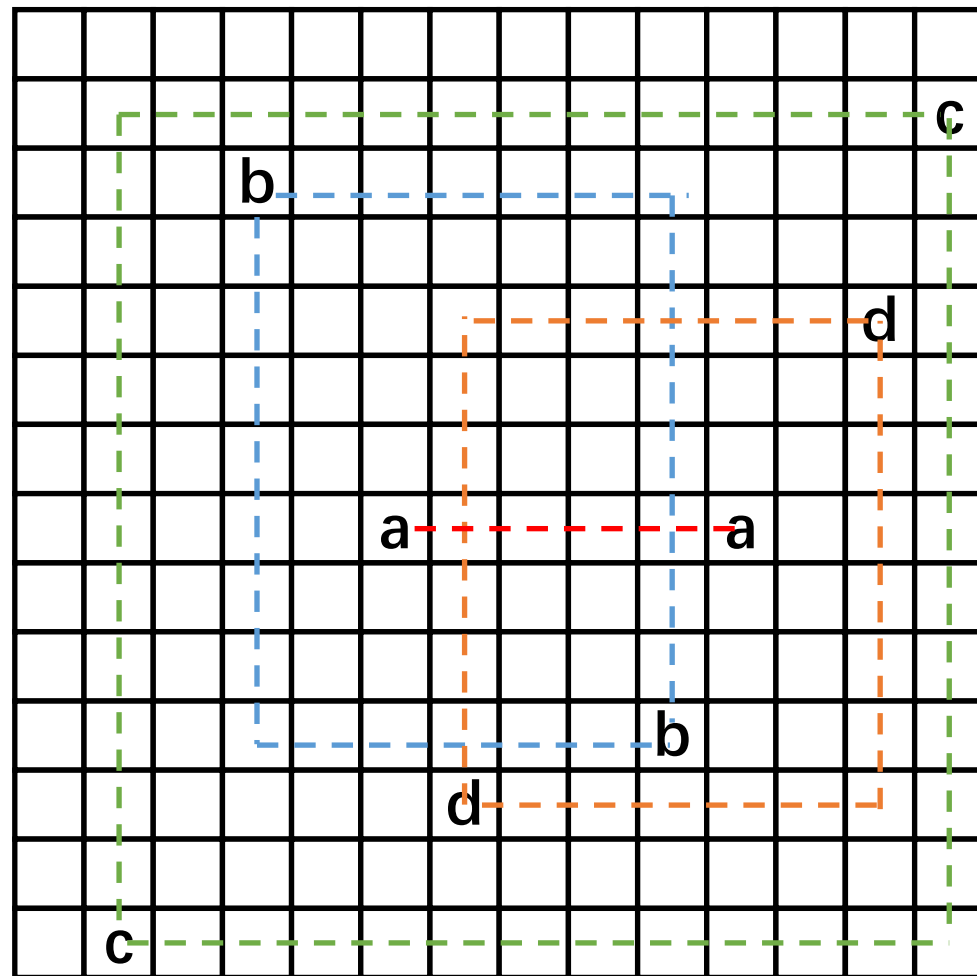


■ 全局布线算法

□ 顺序全局布线

– 启发式的排序

- 如右图所示，首先找出各个线网的覆盖范围
- 得到a的覆盖范围内引脚数量为0，b的为1，d的为2，c的为6
- 所以，线网顺序为a, b, d, c

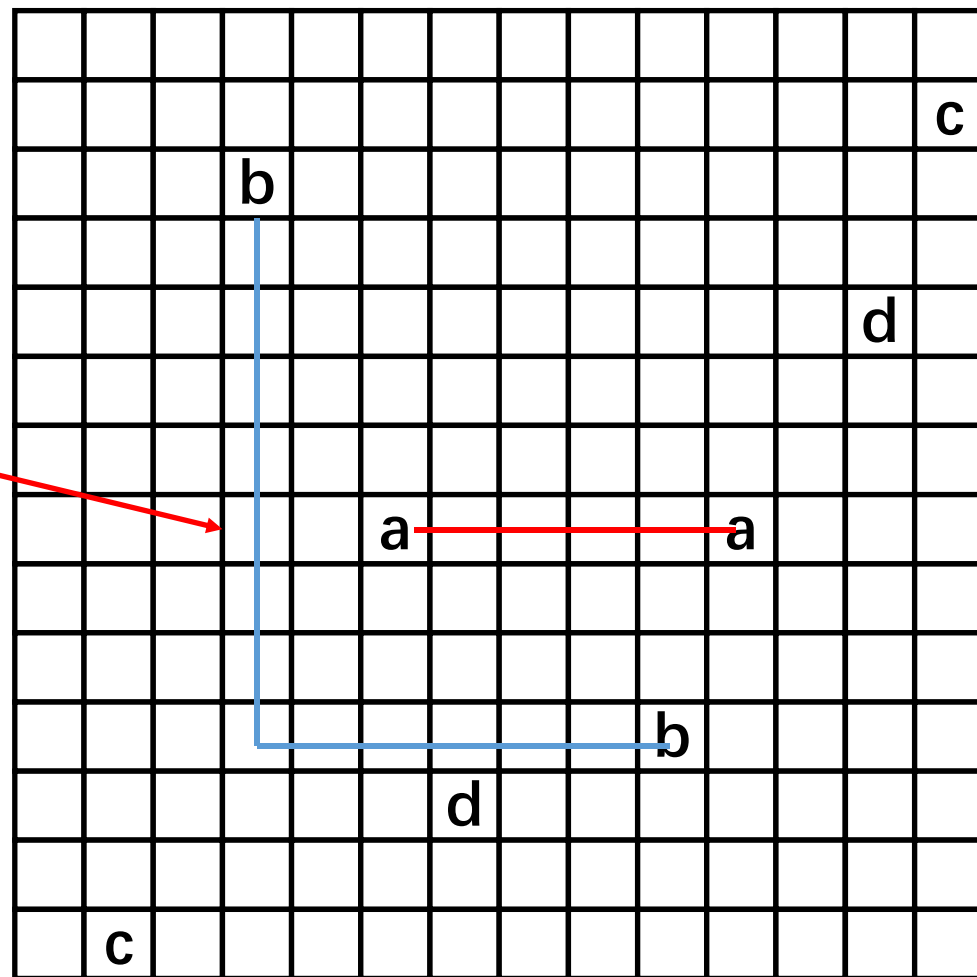


■ 全局布线算法

□ 顺序全局布线

– 启发式的线网排序

- 得到的线网顺序为a, b, d, c
- 首先布线a
- 然后布线b

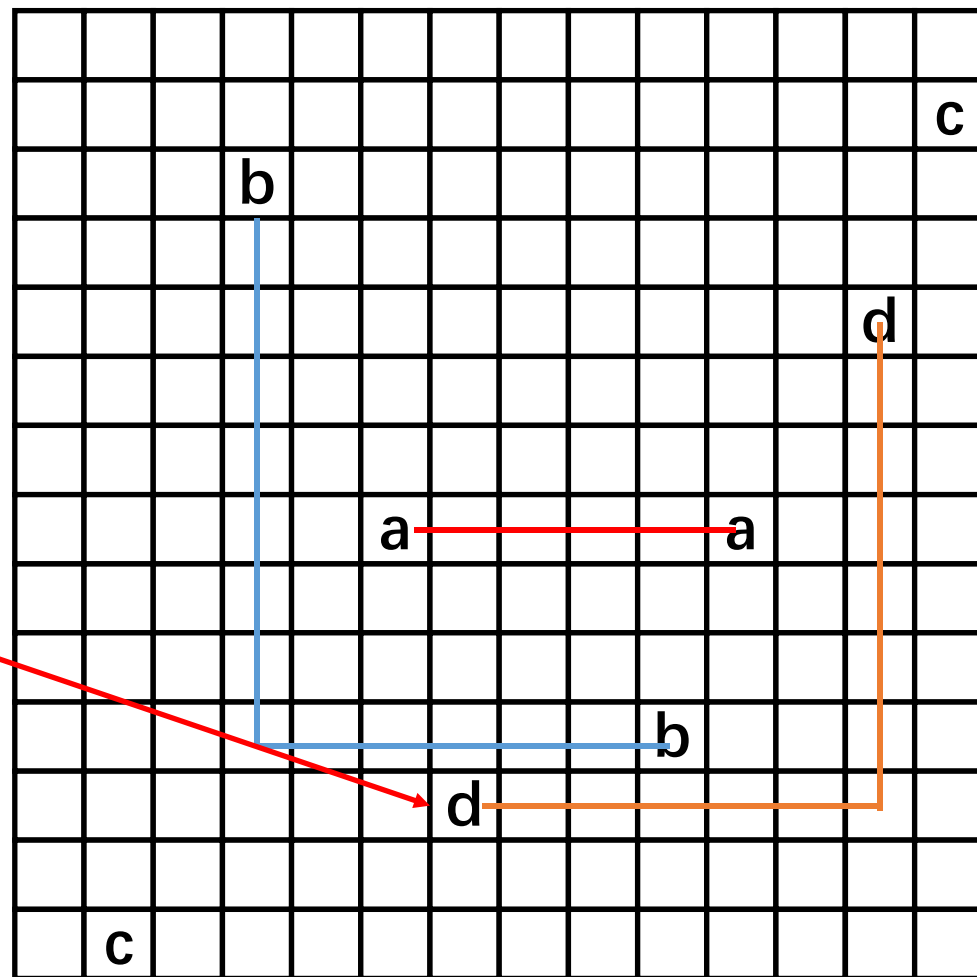


■ 全局布线算法

□ 顺序全局布线

– 启发式的线网排序

- 得到的线网顺序为a, b, d, c
- 首先布线a
- 然后布线b
- 接着布线d

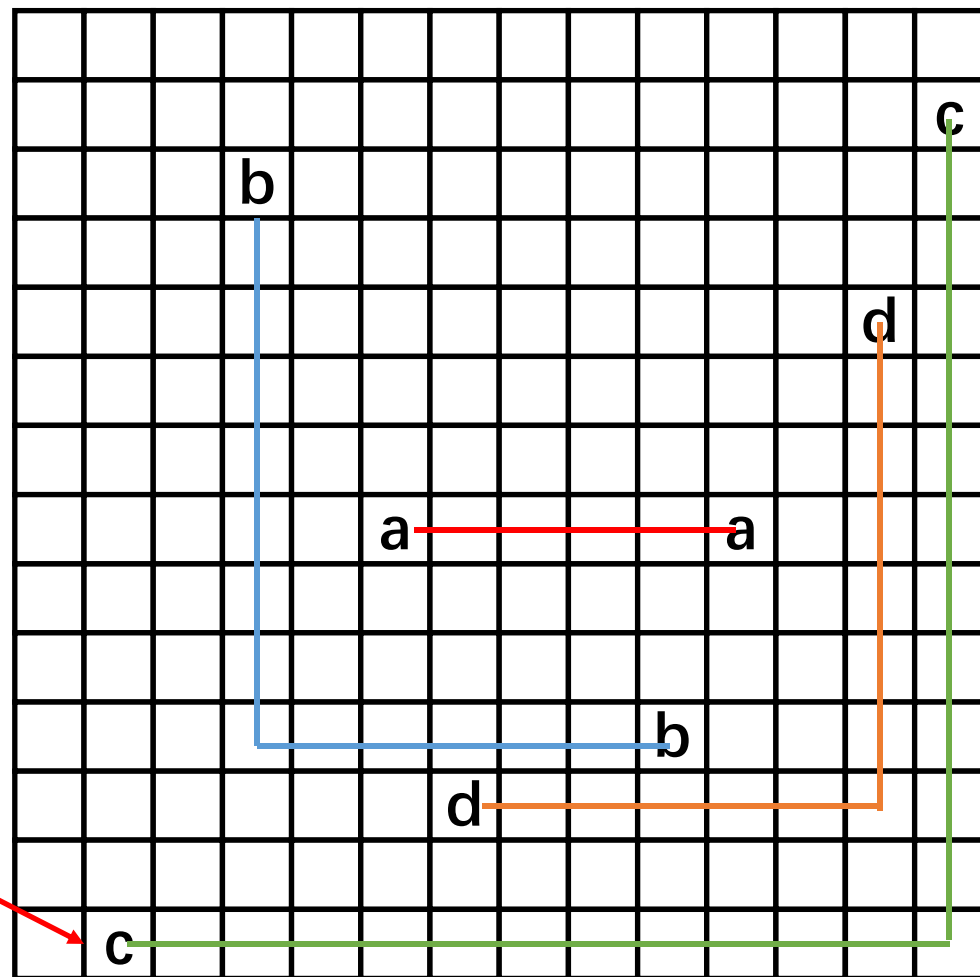


■ 全局布线算法

□ 顺序全局布线

– 启发式的线网排序

- 得到的线网顺序为a, b, d, c
- 首先布线a
- 然后布线b
- 接着布线d
- 最后布线c



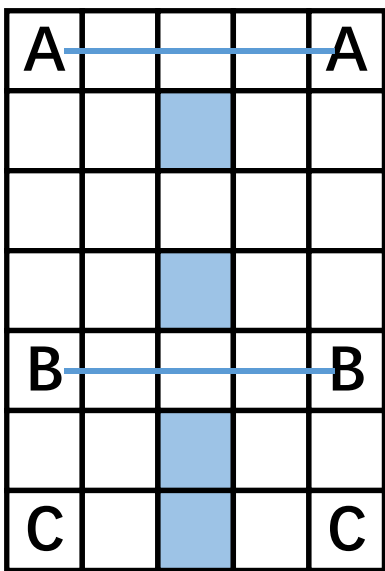
■ 全局布线算法

□ 顺序全局布线

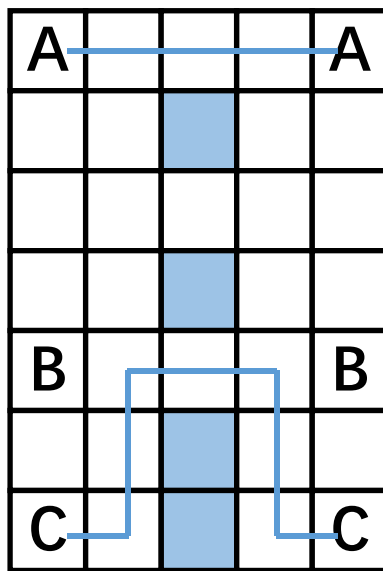
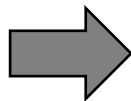
– 拆线与重新布线过程——包含两个步骤

- 如果一些线网布线失败，那么拆除阻塞部分的线网
- 然后先布线被阻塞的线网，再重新布线被拆除的线网

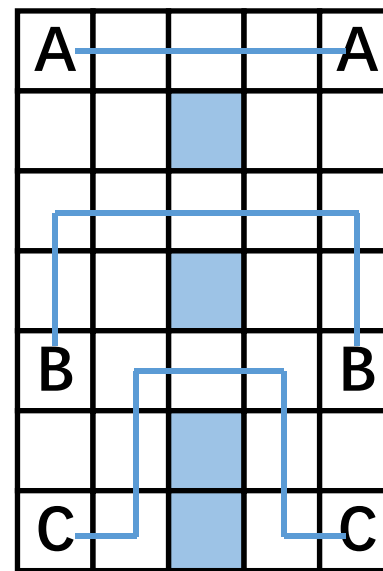
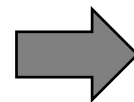
– 以下例子中，按照A, B, C的顺序布线



无法布线C



拆除B的布线，然后布线C



最后布线B

■ 全局布线算法

□ 顺序全局布线

– 优点

- 简单易实现

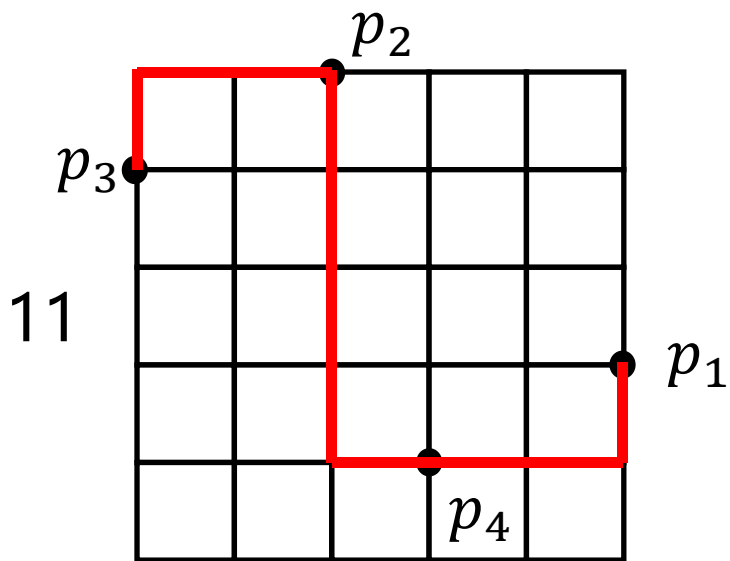
– 缺点

- 布线质量取决于排序好坏
- 在任何排序下，后布线的线网都更难以布线，因为它们受到更多的阻碍
- 如果顺序布线算法未能找到一个可行解，不能确定是因为布局导致不存在可行解，还是因为排序不当导致找不到可行解
- 此外，顺序布线不一定是最优解

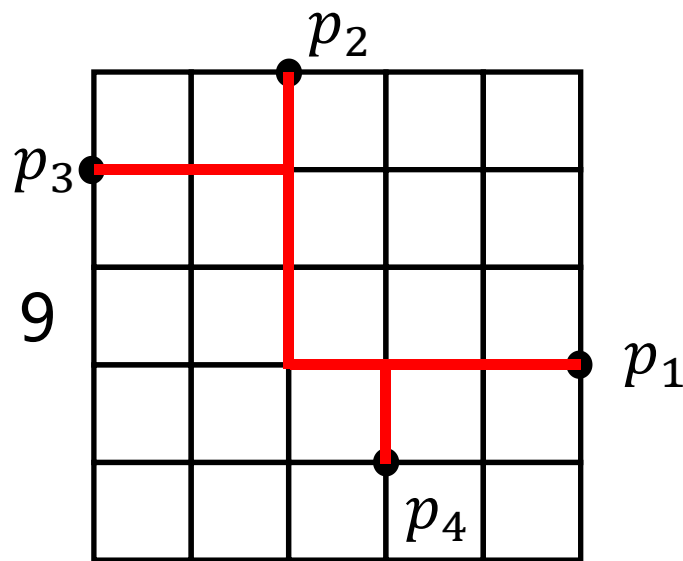
■ 全局布线算法

□ 多引脚线网布线

- 之前介绍的迷宫布线算法，线搜索算法主要是针对两引脚线网的
- 而对于三引脚或更多引脚的线网，一种简单的方法是将每个线网分解为多组两引脚连接，但是这种方法得到的布线结果较差。
- 如下图(a)所示，得到的布线长度为11，不如图(b)的布线结果，线长为9



(a)

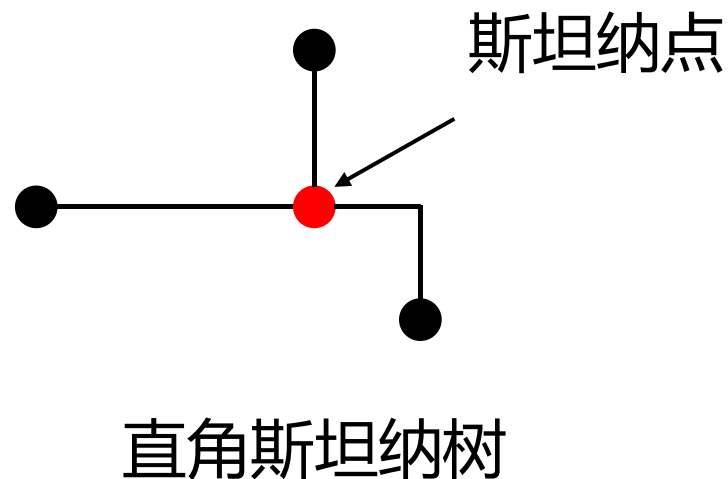
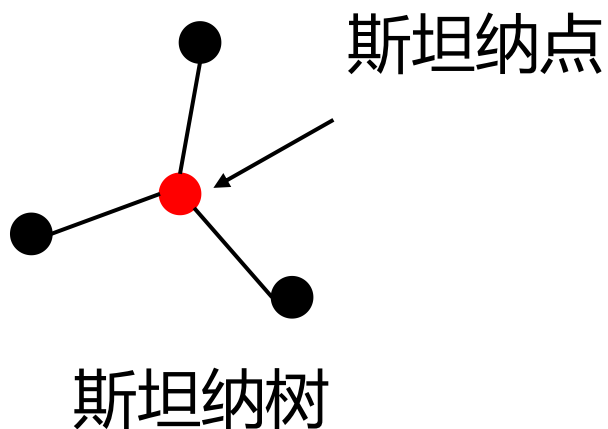


(b)

■ 全局布线算法

□ 多引脚线网与斯坦纳树

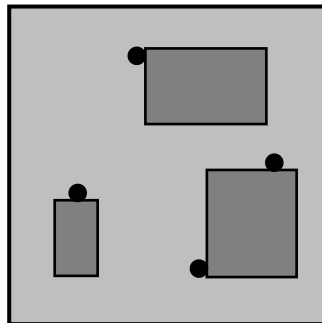
- 斯坦纳树(Steiner Tree)是一种求互联多个顶点最短连接的算法
- 直角斯坦纳树是所有边都水平或垂直的斯坦纳树
 - 给定平面上的 m 个点，最小直角斯坦纳树通过加入一些额外的连接点（称为斯坦纳点）来实现最短的直角斯坦纳树
 - 但找最小斯坦纳树这个问题本身是NP难的，直角斯坦纳树亦是如此



■ 全局布线算法

□ FLUTE

- 一种基于查找表的最小斯坦纳树搜索算法
- 输入
 - 一个有 n 个引脚的线网
 - 每个引脚的坐标 (x_i, y_i)
- 输出
 - 给定线网的最小生成树的线长



■ 全局布线算法

□ FLUTE——算法总览

- 按照问题规模分成两类进行处理
- 如果线网的引脚数量小于等于9个
 - 根据当前线网引脚分布作为索引，查找表格，找到对应的潜在最优线长向量和最优斯坦纳树
 - 依次计算每个潜在最优线长向量对应的布线总线长，返回其中最短的线长
- 如果线网的引脚数量大于9个
 - 不断拆分为小线网，直到可以运用查找表方法

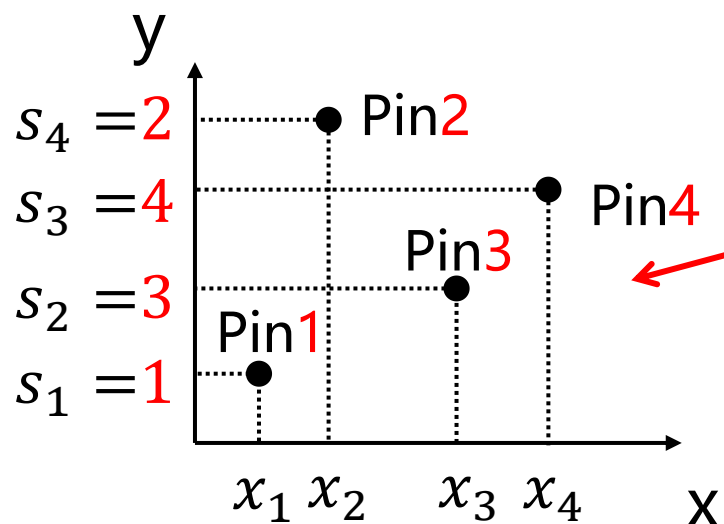
■ 全局布线算法

□ FLUTE——构建搜索索引

– 垂直序列

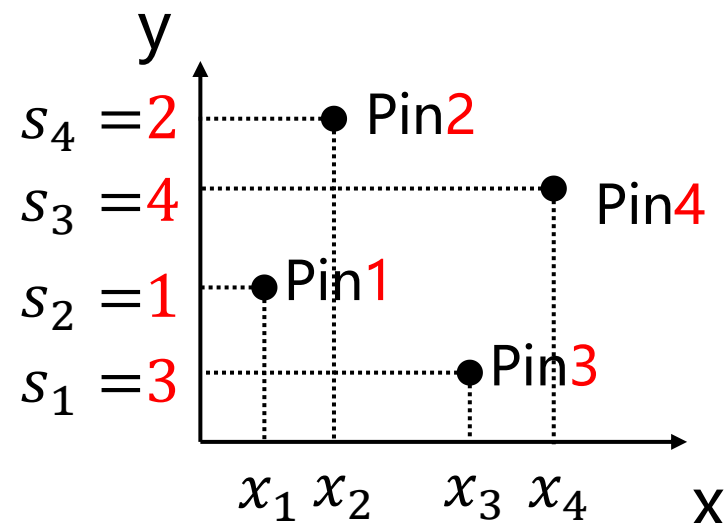
- 引脚名称按 x 坐标升序排序
- 垂直序列 $s_1s_2\dots s_n$ 为按 y 坐标排序的引脚名称

– 这样，不同的垂直序列表示不同的相对位置关系



垂直序列=1342

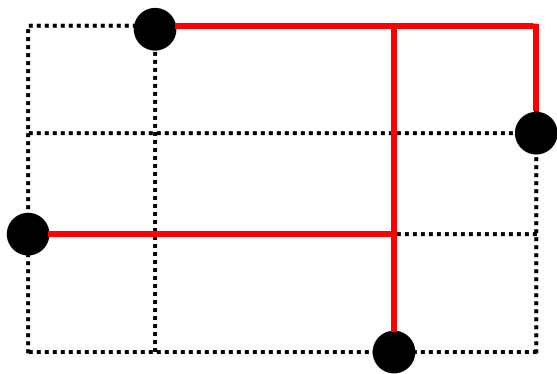
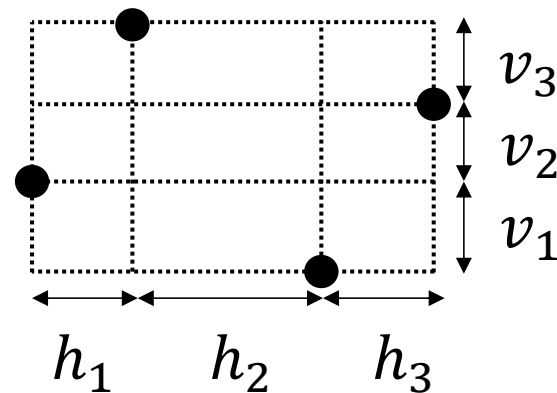
垂直序列=3142



■ 全局布线算法

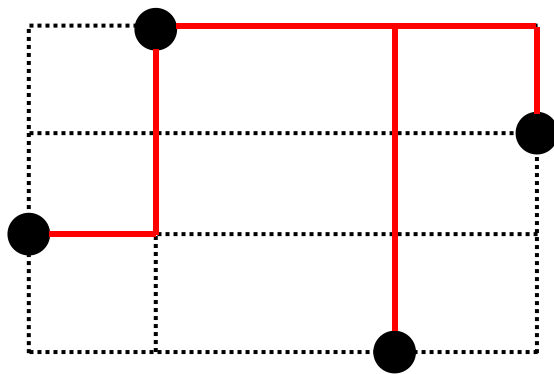
□ FLUTE——线长向量

- 生成树可以表示为边长的线性组合，其中系数为正整数
- 线长向量由对应的系数组成



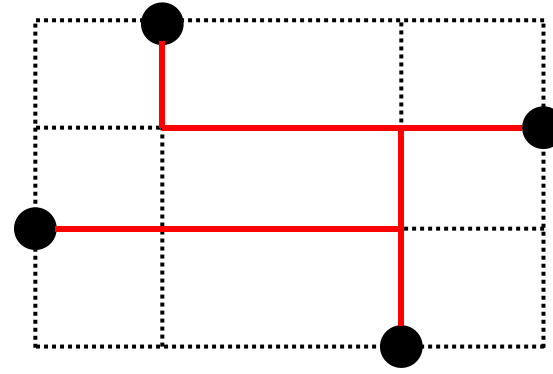
$$WL = h_1 + 2h_2 + h_3 + v_1 + v_2 + 2v_3$$

$$(1, 2, 1, 1, 1, 2)$$



$$WL = h_1 + h_2 + h_3 + v_1 + 2v_2 + 3v_3$$

$$(1, 1, 1, 1, 2, 3)$$



$$WL = h_1 + 2h_2 + h_3 + v_1 + v_2 + v_3$$

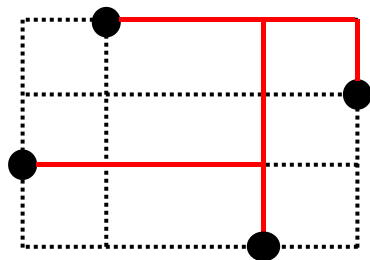
$$(1, 2, 1, 1, 1, 1)$$

■ 全局布线算法

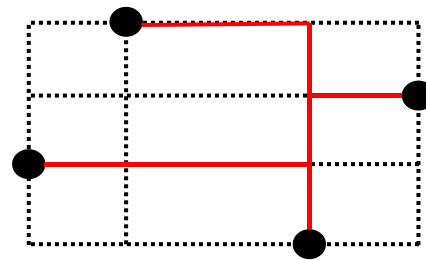
□ FLUTE——潜在最优线长向量

– 潜在最优线长向量(potentially optimal wirelength vector, POWV)是可能产生最优线长的线长向量

- 大多数线长向量不是最优线长向量，甚至都不是潜在最优的。一个线长向量必须是最小的才有可能成为潜在最优线长向量
- 因此可以剔除很多无用解



(1,2,1,1,1,2)



(1,2,1,1,1,1)

(1,2,1,1,1,2)是多余向量，它并不是满足连接关系的所有向量中最小的一种。

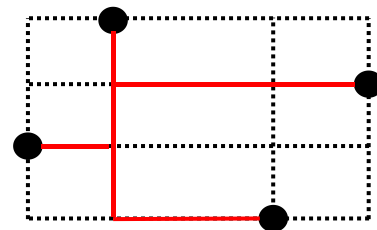
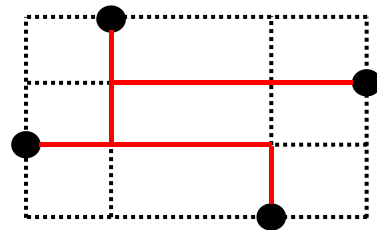
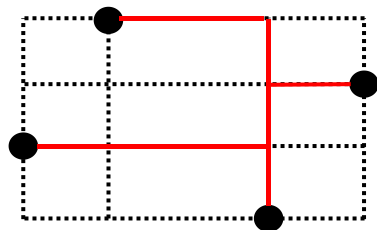
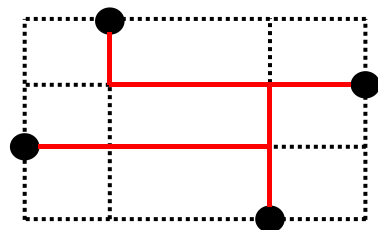
(1,2,1,1,1,1)是同一系列向量中最小的向量，是潜在最优线长向量

■ 全局布线算法

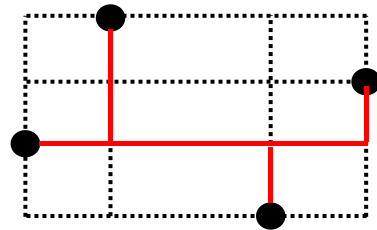
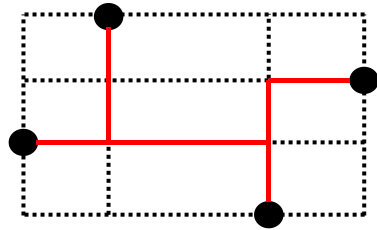
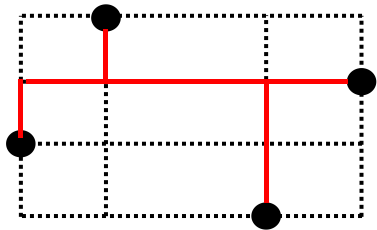
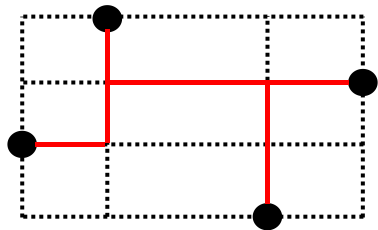
□ FLUTE——潜在最优线长向量

- 对于任何线网
 - 布线解的数量可能很大
 - 其中相当多的解是冗余的，但潜在最优线长向量的数量是十分少的
- 以下8种布线解，可以被2个潜在最优线长向量表示

(1,2,1,1,1,1)



(1,1,1,1,2,1)

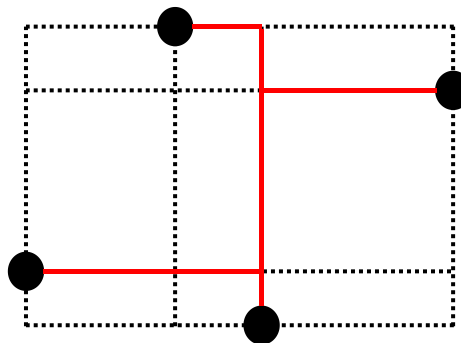
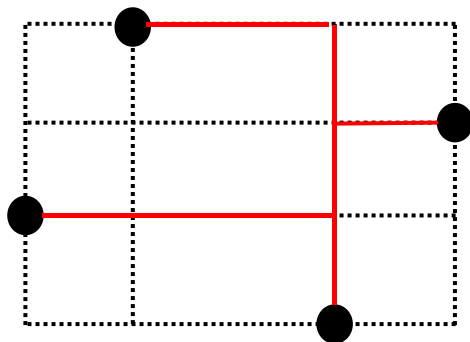


■ 全局布线算法

□ FLUTE——潜在最优线长向量

– 不同线网可以共享潜在最优线长向量

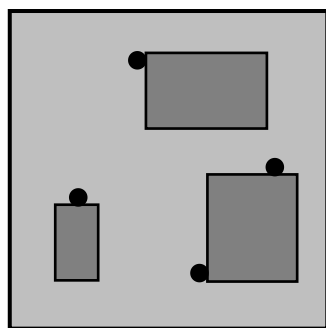
- 预先计算所有潜在最优线长向量和对应的一个斯坦纳树，并将其存储在查找表中
- 引脚的位置有无限种可能——计算并存储并不现实
- 拓扑等价的斯坦纳树合并为一组
 - **拓扑等价**：仅通过调整 $h_1, h_2, h_3, v_1, v_2, v_3$ 使得两个斯坦纳树一致
 - 如下图所示的斯坦纳树可以归为一类，它们之间的区别只是 $h_1, h_2, h_3, v_1, v_2, v_3$ 不同



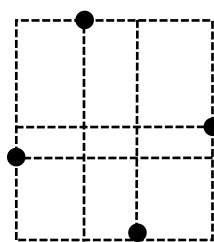
■ 全局布线算法

□ FLUTE线长估计例子步骤1

- 输入一个线网，已知其所有引脚的坐标
- 根据引脚坐标 x_i, y_i 插入Hanan网格线



线网引脚

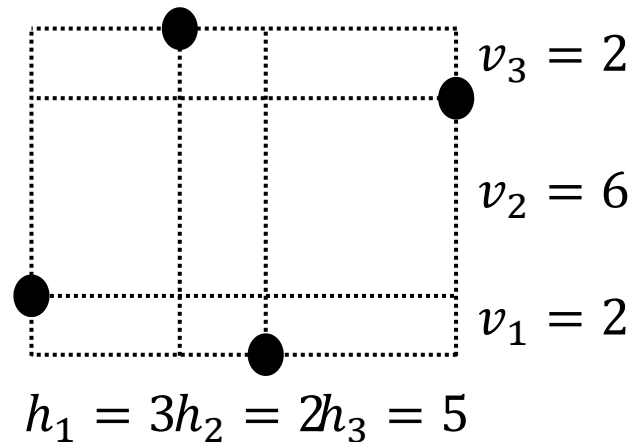
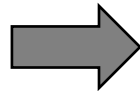
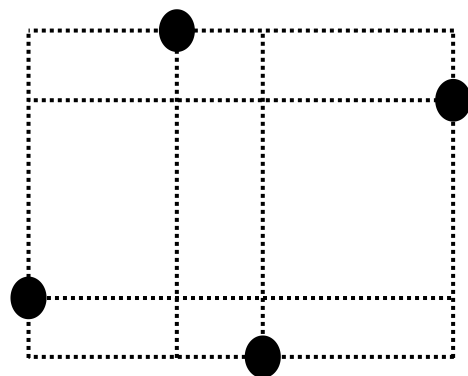


插入网格线
构成Hanan网格

■ 全局布线算法

□ FLUTE线长估计例子步骤2

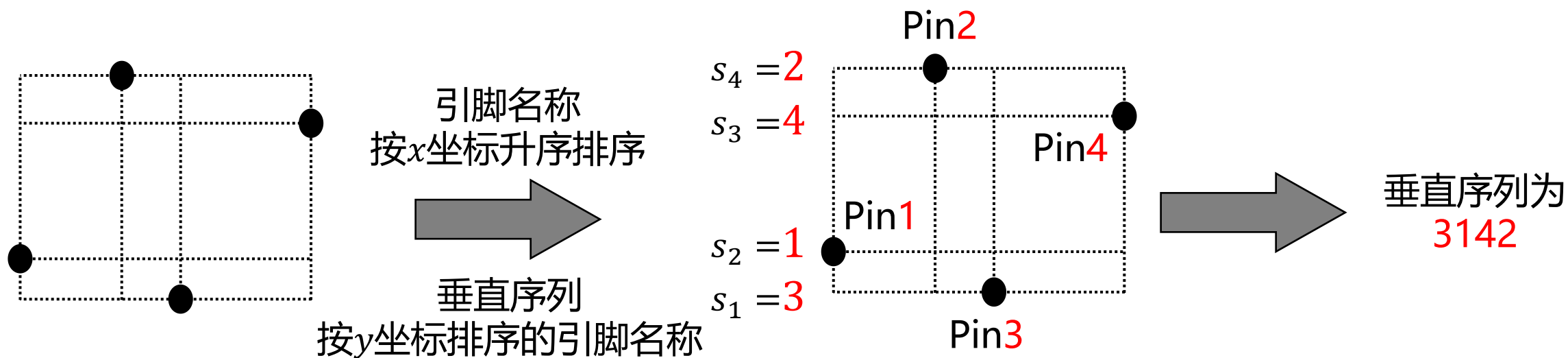
– 确定网格线的 h_i 和 v_i



■ 全局布线算法

□ FLUTE线长估计例子步骤3

– 确定线网的垂直序列

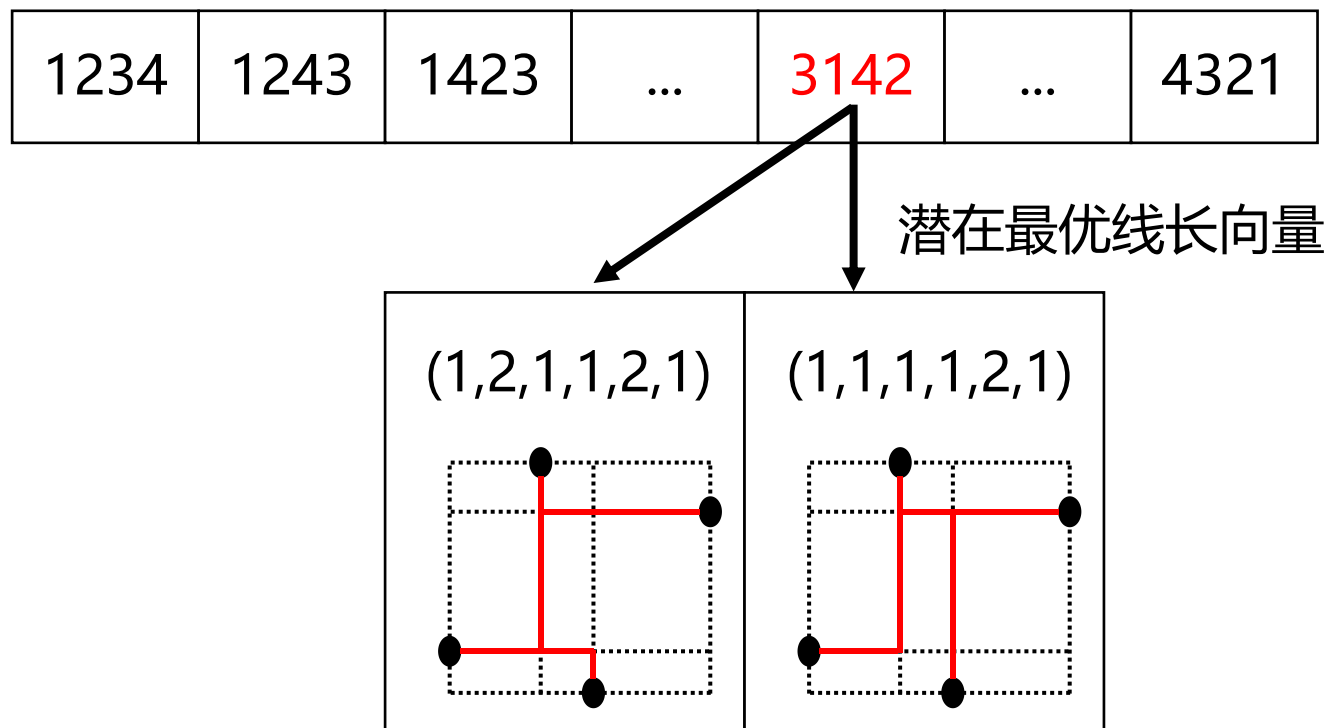


■ 全局布线算法

FLUTE线长估计例子步骤4

- ## — 根据垂直序列检索对应潜在最优线长向量

查找表索引



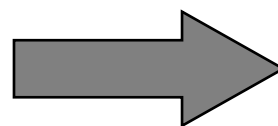
■ 全局布线算法

□ FLUTE线长估计例子步骤5

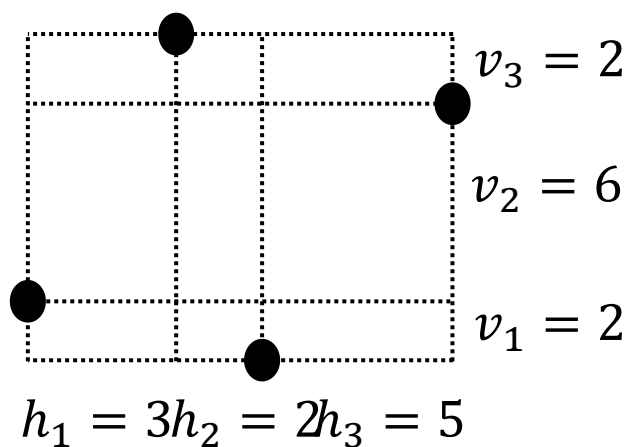
- 计算每个潜在最优线长向量的线长，并返回最优线长

$$(1,2,1,1,1,1) \Rightarrow h_1 + 2h_2 + h_3 + v_1 + v_2 + v_3 = 22$$

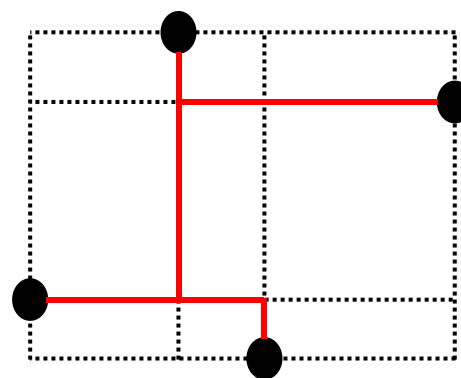
$$(1,1,1,1,2,1) \Rightarrow h_1 + h_2 + h_3 + v_1 + 2v_2 + v_3 = 26$$



最优线长为22



对应的斯坦纳树



■ 全局布线算法

□ FLUTE查找表方法总结

– 优点

- 在度数小的 (≤ 7) 线网上很有用, 效率高且精度高

– 缺点

- 查找表对于高度数线网 (≥ 9) 不切实际
 - 网格的垂直序列数量与网格的度数是阶乘关系
 - 度为9需要存 $362880 \times 30.039 \approx 10900552$ 个潜在最优线长向量, 存储和查找这么多向量十分困难
- 需要使用递归分治+启发式的方法分解高度数的线网, 牺牲解质量

线网度	垂直序列数量	每个垂直序列 潜在最优线长向量数量		
		最小值	平均值	最大值
2	2	1	1	1
3	6	1	1	1
4	24	1	1.667	2
5	120	1	2.467	3
6	720	1	4.433	8
7	5040	1	7.932	15
8	40320	1	15.251	33
9	362880	1	30.039	79

PART 03

详细布线算法

■ 详细布线算法

□ 通道布线 (Channel routing)

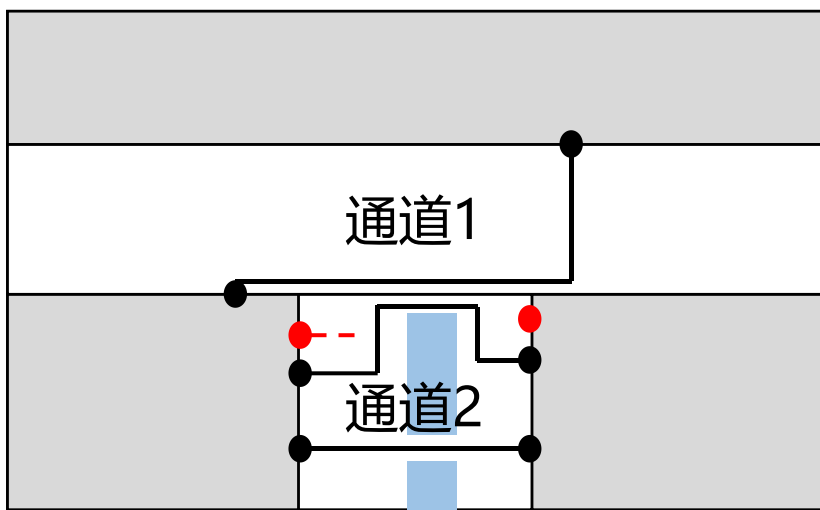
- 通道布线是一种最常见的详细布线类型
- 通道布线是一种特殊类型的布线问题，其中导线连接在布线通道内完成。为了应用通道布线，布线区域通常被分解为多个布线通道。分解布线区域的方式往往有多种，下图展示了T形布线区域的两种分解方式



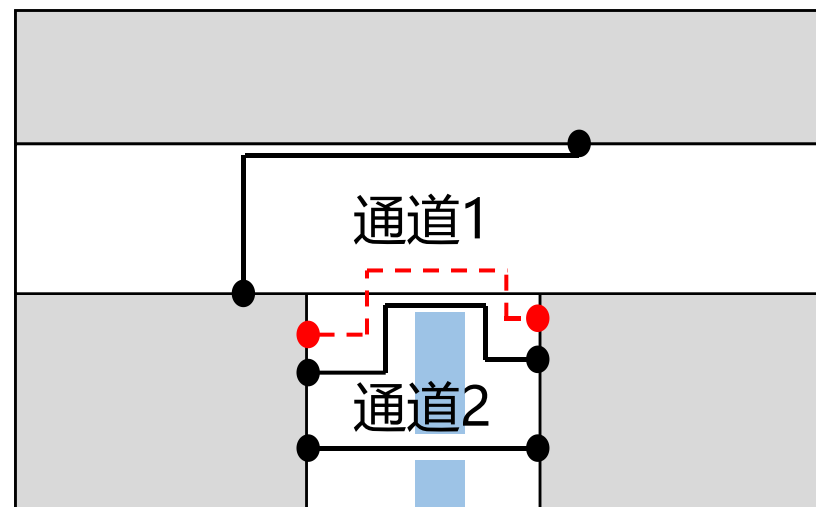
■ 详细布线算法

□ 通道布线

- 布线区域的顺序对通道布线过程有很大影响
- 在下图中，如果按照先布线通道2再布线通道1的顺序进行布线，则不会发生冲突
- 然而，如果先布线通道1并将所有相关布线固定在该通道中，那么如果通道2无法容纳所有线网，则通道2布线会失败



如果先布线通道1，再布线通道2
通道2无法扩展，红色引脚无法布线



如果先布线通道2，再布线通道1
红色引脚布线可扩展至通道1

■ 详细布线算法

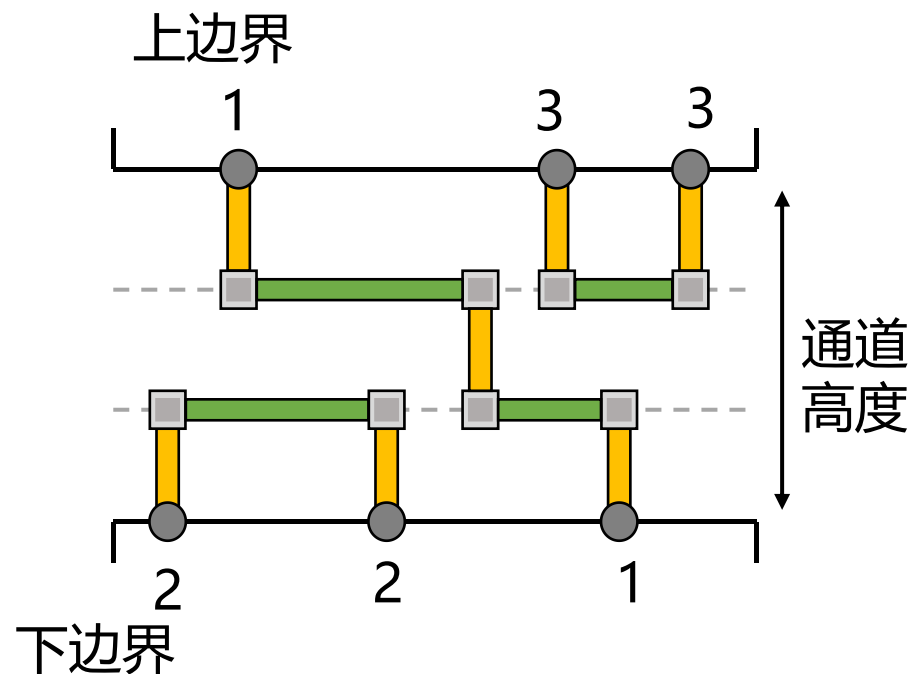
□ 通道布线

— 输入

- 两个通道边界——上边界和下边界
- 边界的列上有引脚编号，具有相同编号的引脚属于同一个线网，需要相互连接

— 目标

- 最小化通道高度——关系芯片尺寸和制造成本

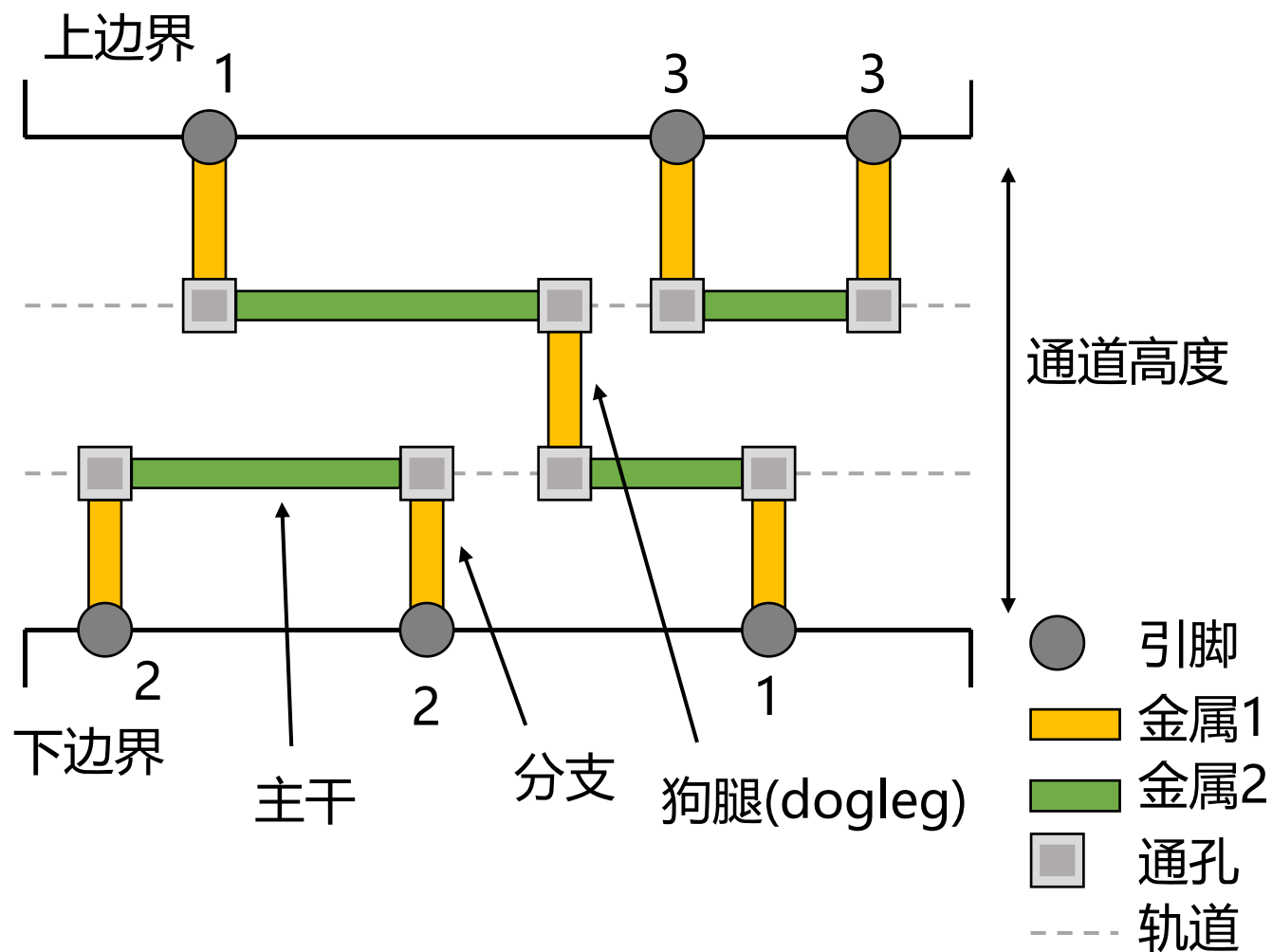


■ 详细布线算法

□ 通道布线

– 右图展示了一个两层通道布线的例子

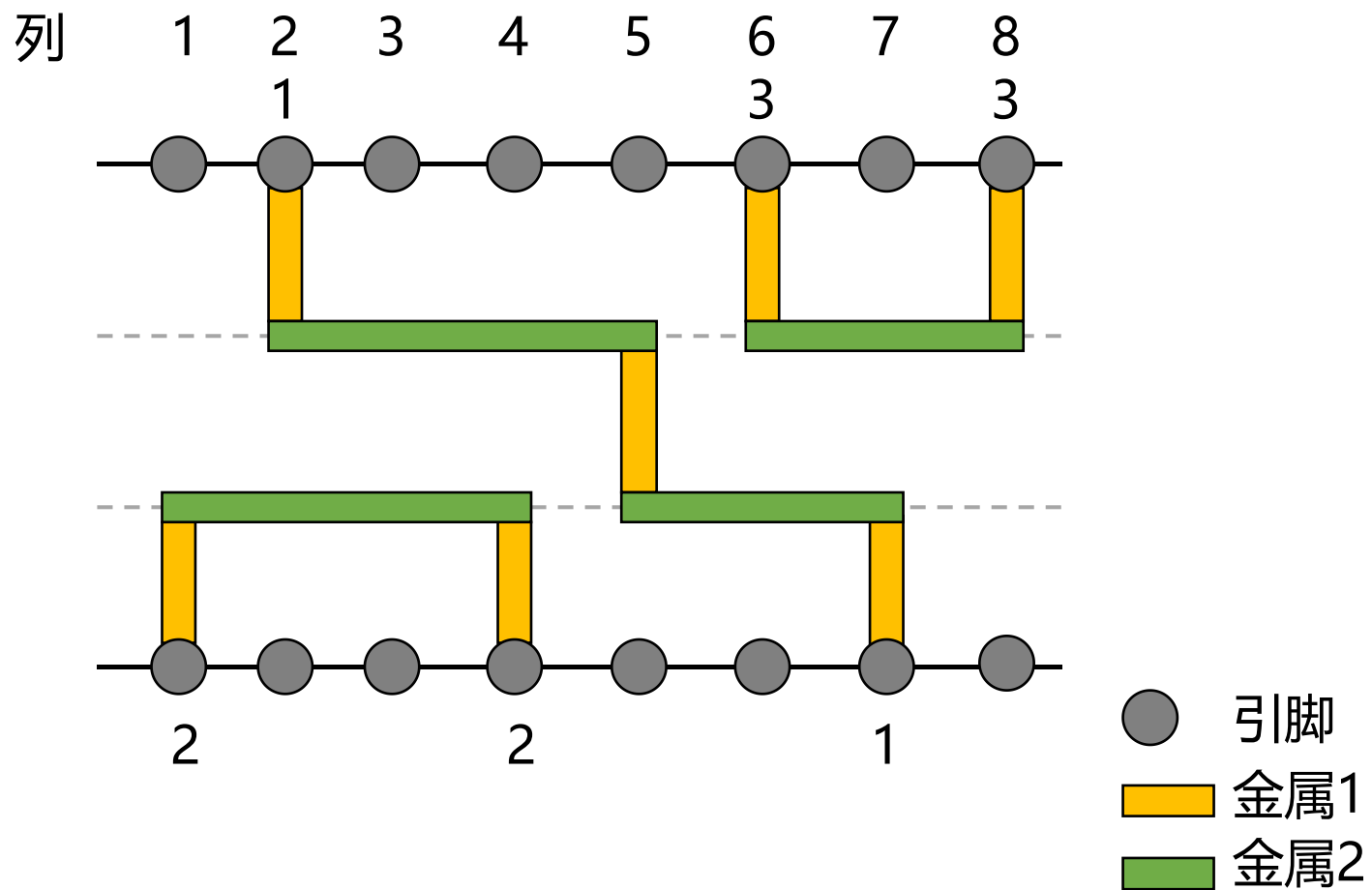
- 连接线网1, 2, 3
- 在轨道上的水平导线段称为主干
- 用于连接主干与引脚的垂直导线段称为分支
- 如果某个线网的布线路径包含多个主干, 则该布线路径称为狗腿布线
 - 例如1的连接是一个狗腿布线(dogleg)



■ 详细布线算法

□ 通道布线

- 为了简洁起见，将通道布线的结果图使用右图所示的简化图，省略了引脚的表示

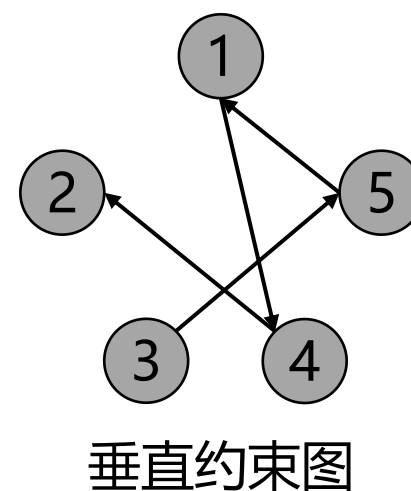
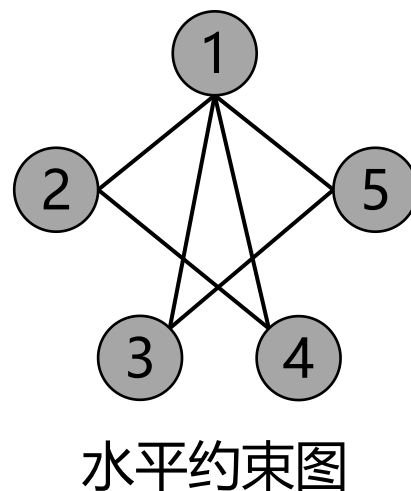
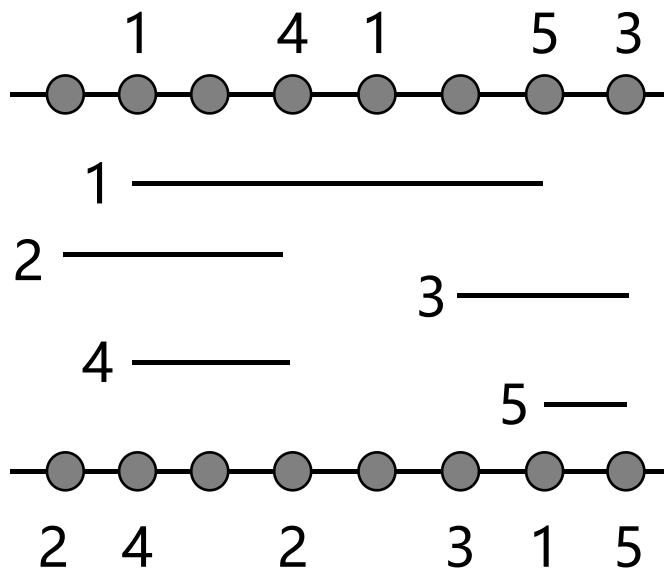


■ 详细布线算法

□ 约束图

- 包含水平约束图和垂直约束图
- 水平约束图和垂直约束图可以表示线网之间的相对关系，有无重叠
- 通过水平约束图和垂直约束图可以检测潜在的布线冲突

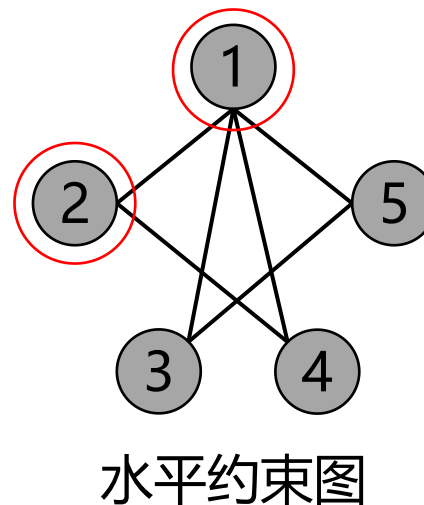
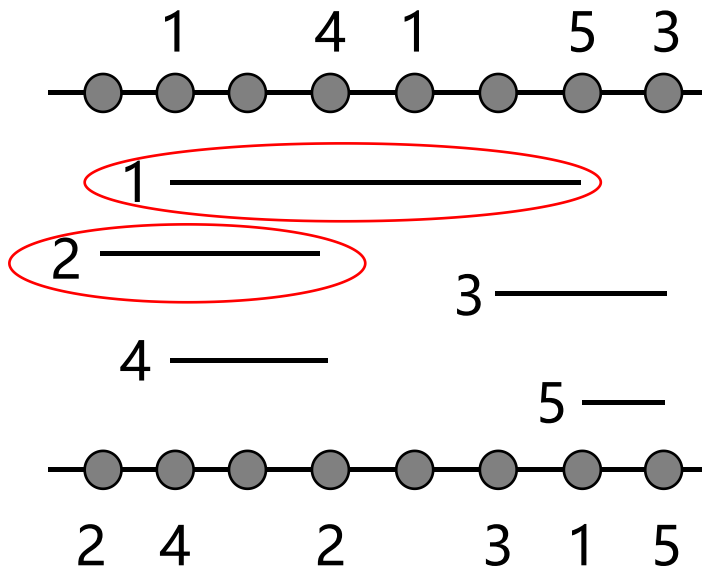
➤ 下图展示了一个通道布线例子的水平约束图和垂直约束图



■ 详细布线算法

□ 水平约束图

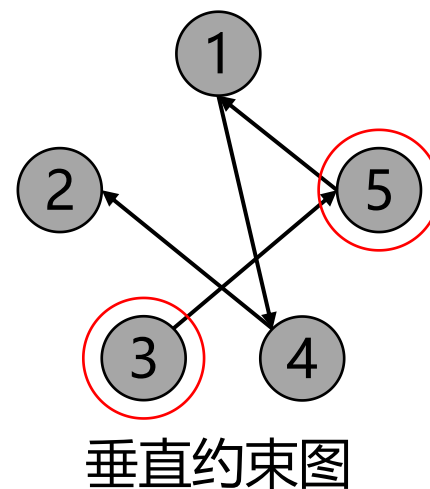
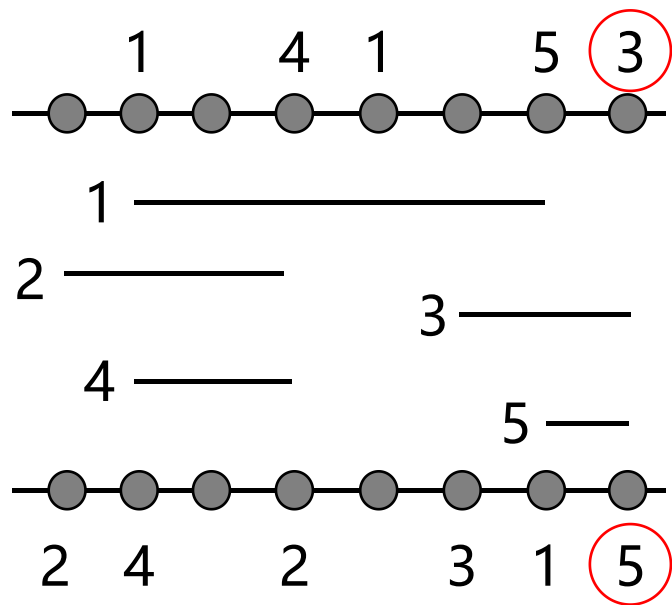
- 水平约束图是一个无向图，如果两个线网 n_i, n_j 的区间有重叠，则说明它们之间存在水平约束，在水平约束图上添加一条边连接 n_i, n_j 。
- 如下图所示，1和2的区间重叠，表明1和2之间存在水平约束，在布线时1和2不能布线在同一轨道上



■ 详细布线算法

□ 垂直约束图

- 垂直约束图是一个有向图，当连接 n_i 的主干在 n_j 的主干之上，即某一列的上边界引脚为 n_i ，下边界引脚为 n_j ，则说明它们之间存在垂直约束，在垂直约束图当中添加一条边连接 n_i 和 n_j
- 如下图所示，3和5之间存在垂直约束，在布线时3的轨道必须在5之上



■ 详细布线算法

□ 左边算法 (Left-edge algorithm)

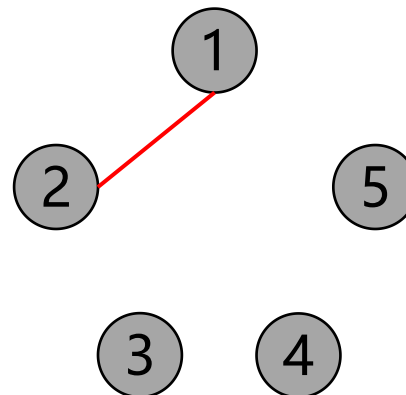
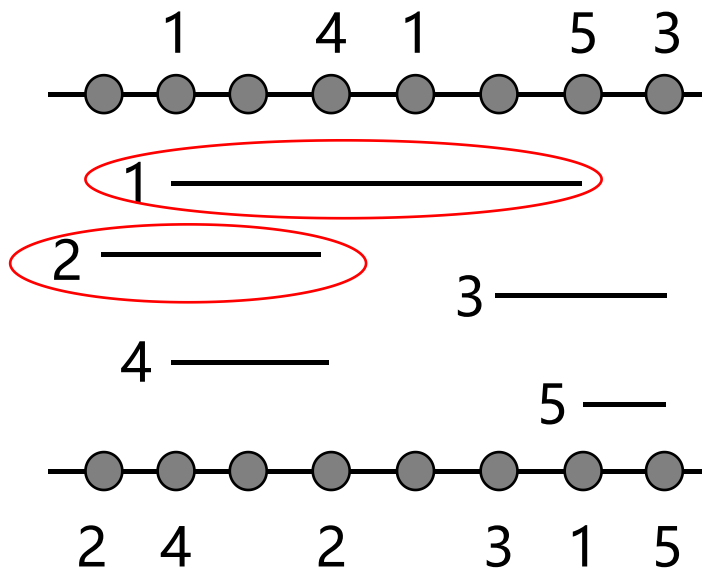
- 包含两个步骤
- 构建约束图：根据这些连接的位置，构建水平约束图和垂直约束图来建模布线约束
- 布线：根据水平约束图和垂直约束图的约束条件布线每个线网

■ 详细布线算法

□ 左边算法例子

– 构建约束图

- 水平约束图是一个无向图，如果两个连接 n_i, n_j 的区间有重叠，则说明它们之间存在水平约束，在水平约束图上添加一条边连接 n_i, n_j 。如下所示，连接1和2的区间有重叠，则在水平约束图中添加一条1到2的边

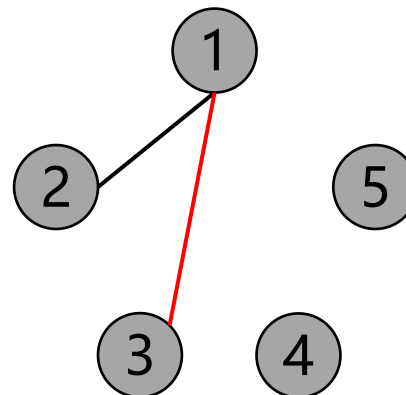
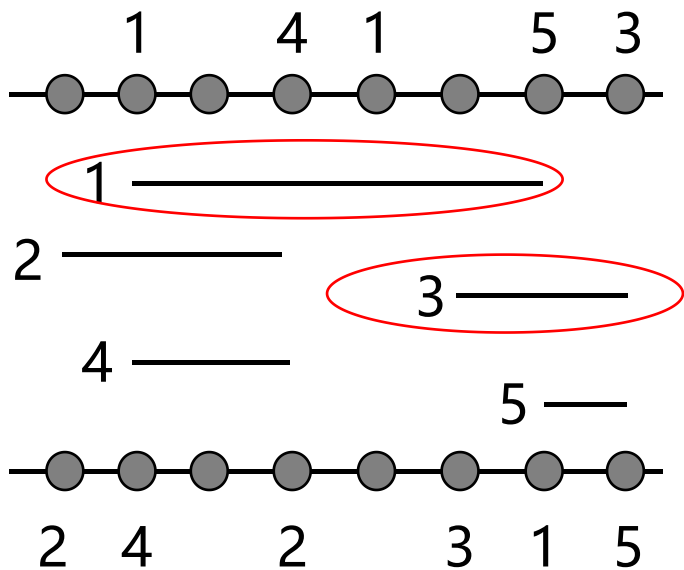


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 如下所示，连接1和3的区间有重叠，则在水平约束图中添加一条1到3的边

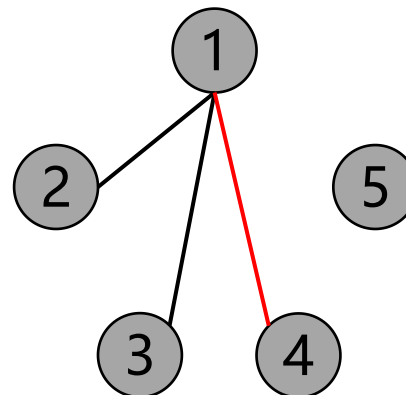
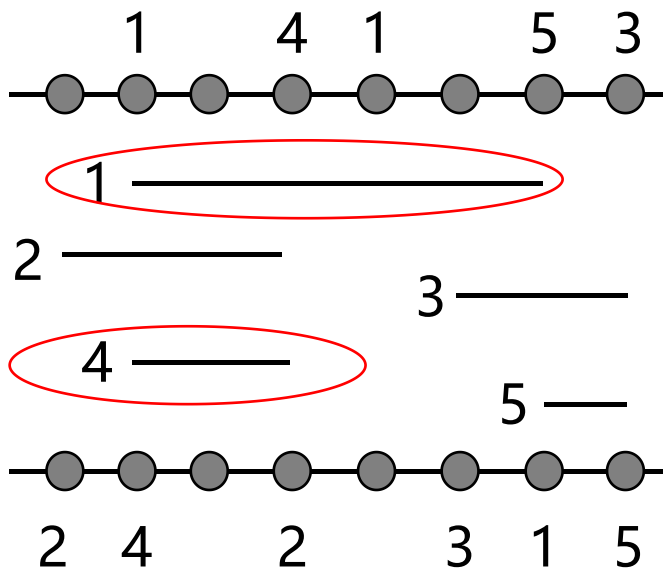


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 如下所示，连接1和4的区间有重叠，则在水平约束图中添加一条1到4的边

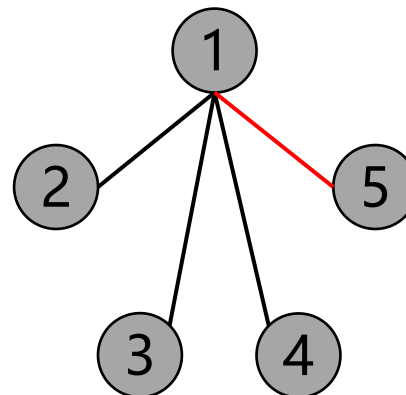
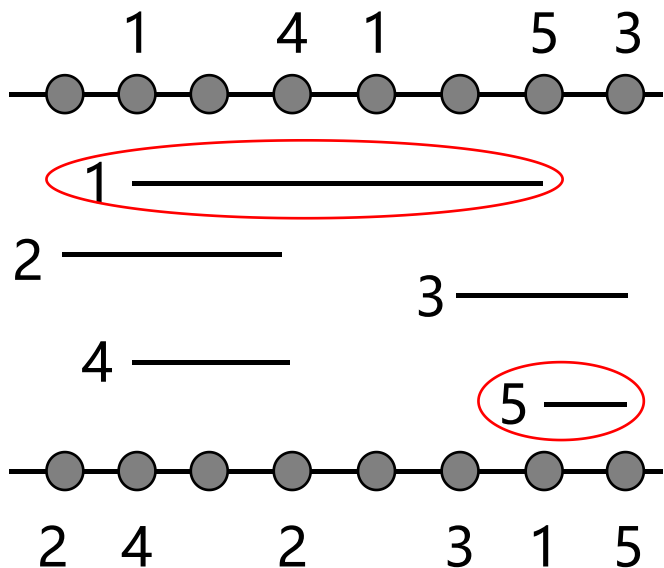


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 如下所示，连接1和5的区间有重叠，则在水平约束图中添加一条1到5的边

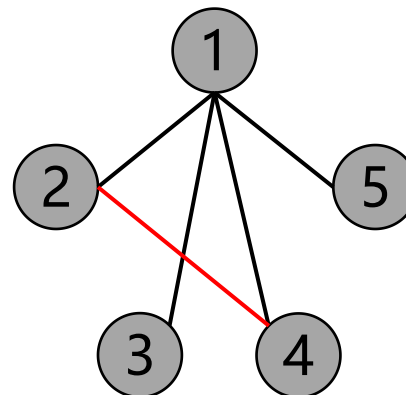
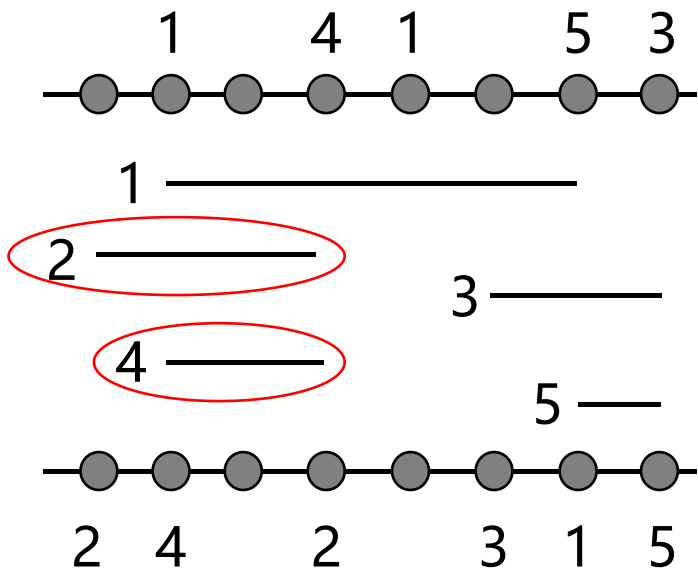


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 如下所示，连接2和4的区间有重叠，则在水平约束图中添加一条2到4的边

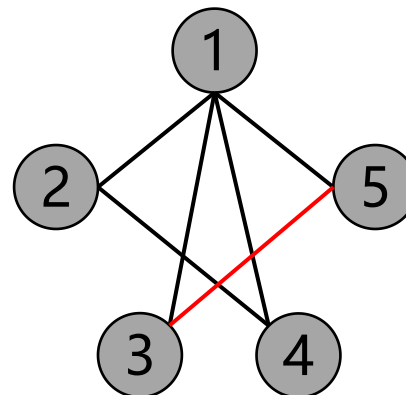
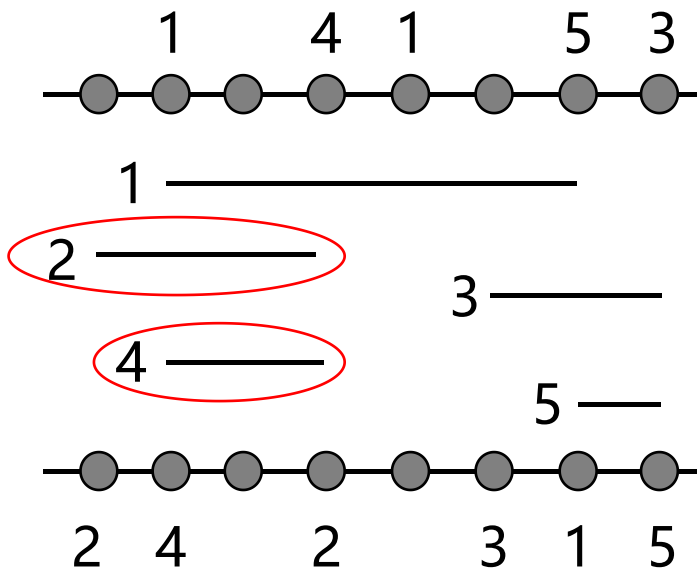


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 如下所示，连接3和5的区间有重叠，则在水平约束图中添加一条3到5的边

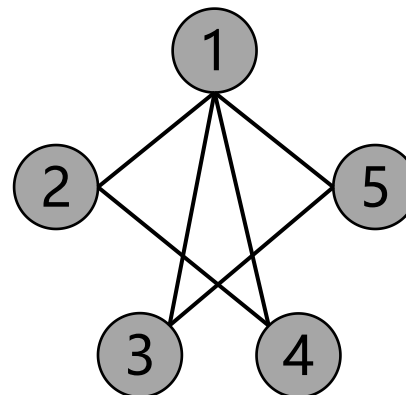
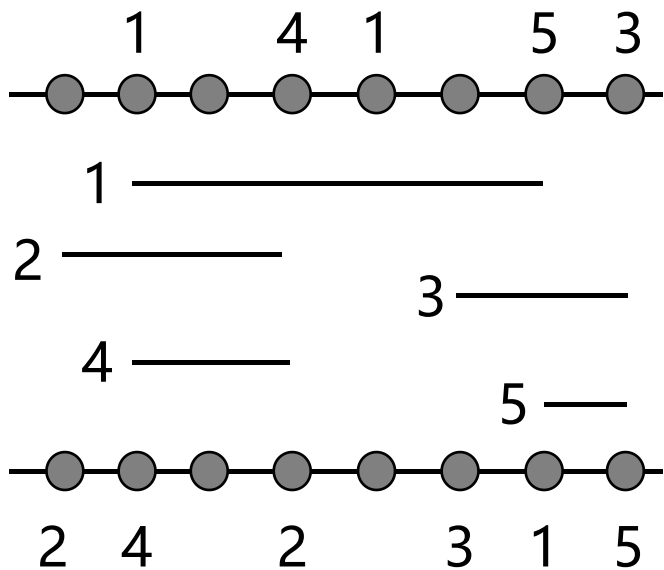


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 按照上述规则构建好水平约束图

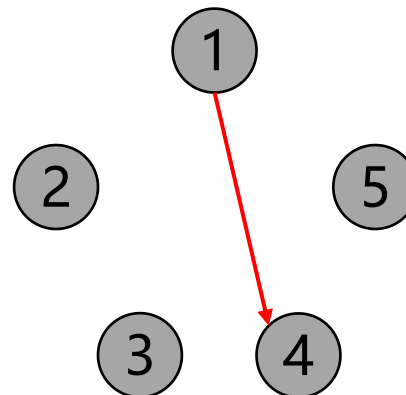
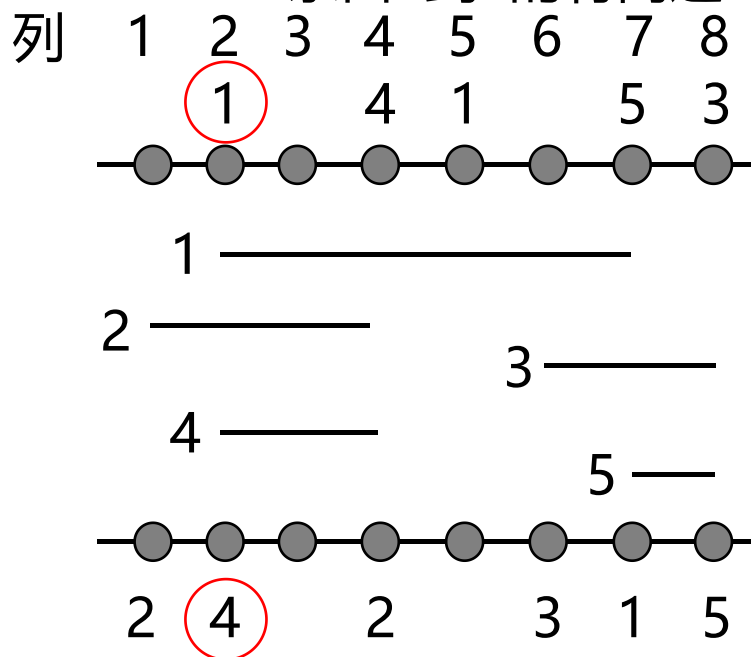


■ 详细布线算法

□ 左边算法例子

– 构建约束图

- 垂直约束图是一个有向图，当连接 n_i 的主干在 n_j 的主干之上，即某一系列的上边界为 n_i ，下边界为 n_j ，则说明它们之间存在垂直约束，在垂直约束图当中添加一条边连接 n_i 和 n_j
- 如下所示，由于第2列上边界和下边界的引脚分别为1和4，所以在垂直约束图中添加一条由1到4的有向边

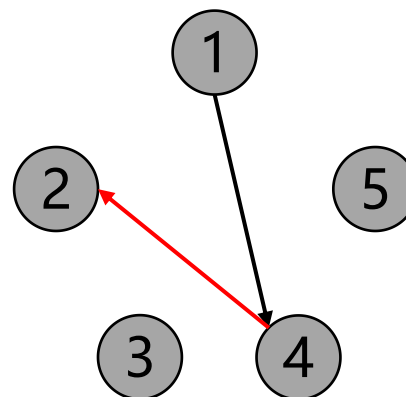
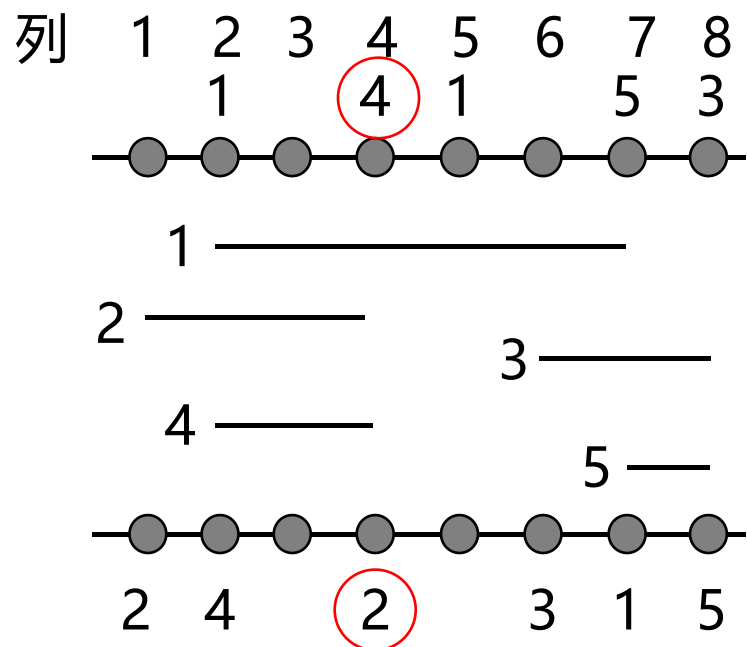


■ 详细布线算法

□ 左边算法例子

– 构建约束图

- 如下所示，由于第4列上边界和下边界的引脚分别为4和2，所以在垂直约束图中添加一条由4到2的有向边

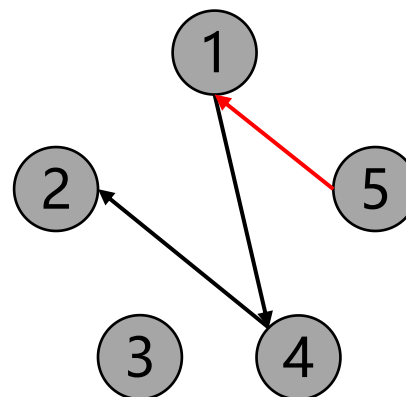
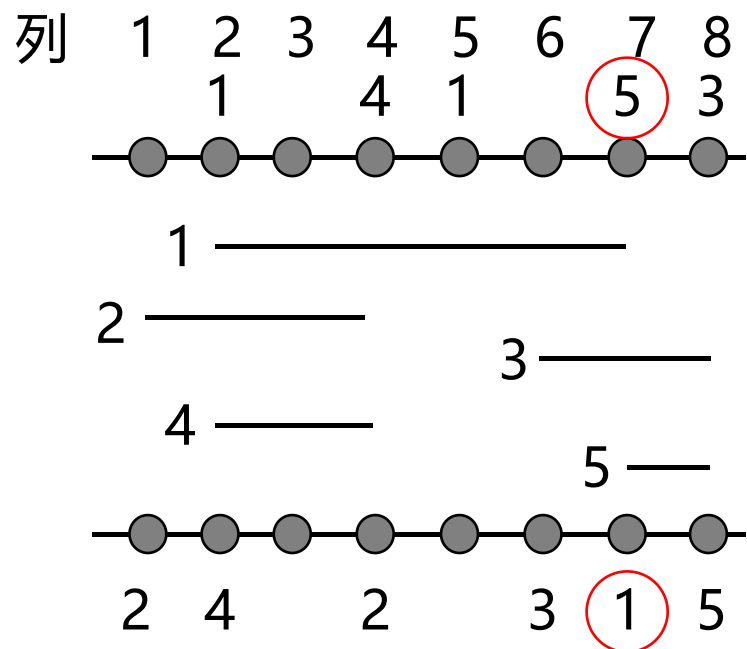


■ 详细布线算法

□ 左边算法例子

– 构建约束图

- 如下所示，由于第7列上边界和下边界的引脚分别为5和1，所以在垂直约束图中添加一条由5到1的有向边

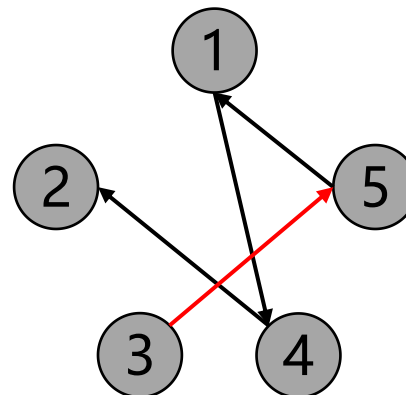
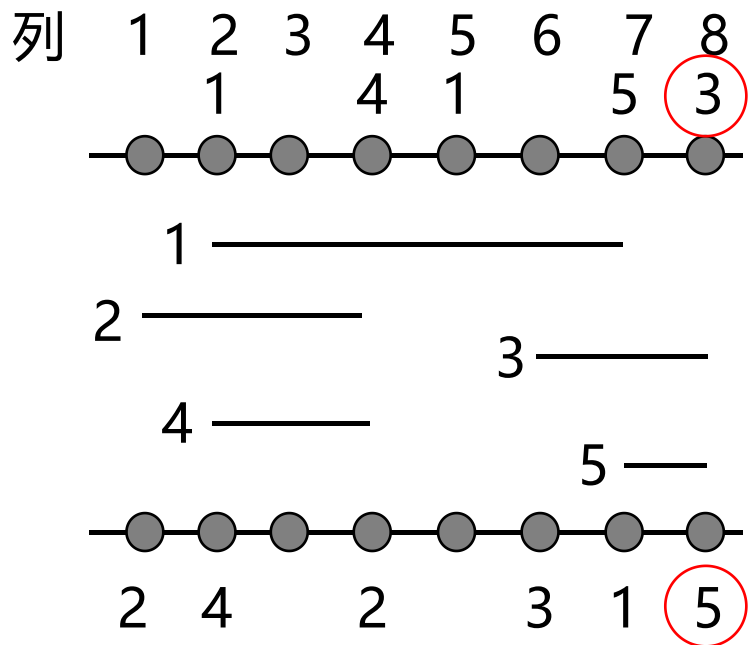


■ 详细布线算法

□ 左边算法例子

– 构建约束图

- 如下所示，由于第8列上边界和下边界的引脚分别为3和5，所以在垂直约束图中添加一条由3到5的有向边

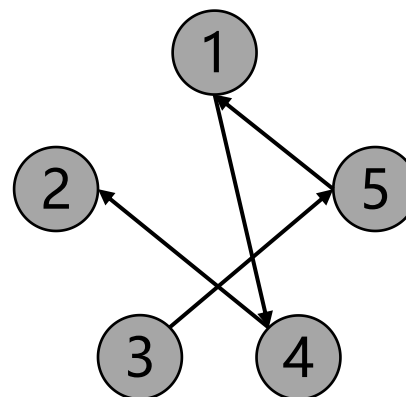
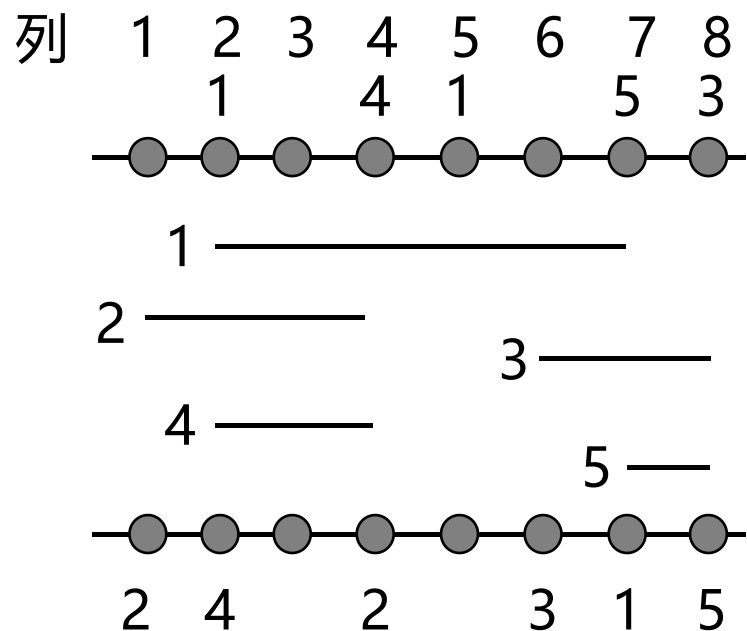


■ 详细布线算法

□ 左边算法例子

– 构建约束图

➤ 按照上述规则构建好垂直约束图

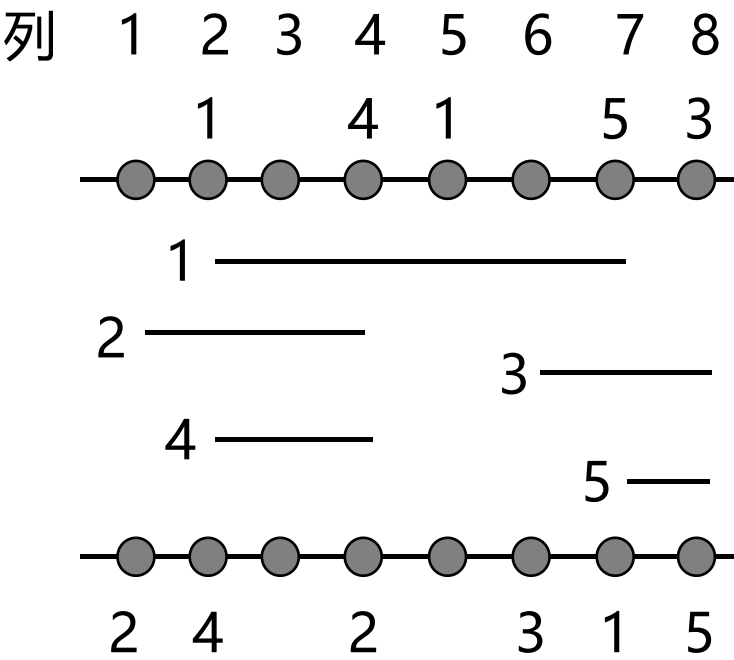


■ 详细布线算法

□ 左边算法例子

– 布线

➤ 首先，将每个连接进行排序，排序方式是根据连接区间的左端坐标进行升序排序



连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]

■ 详细布线算法

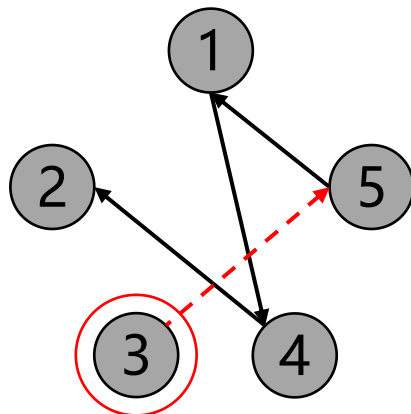
□ 左边算法例子

– 布线

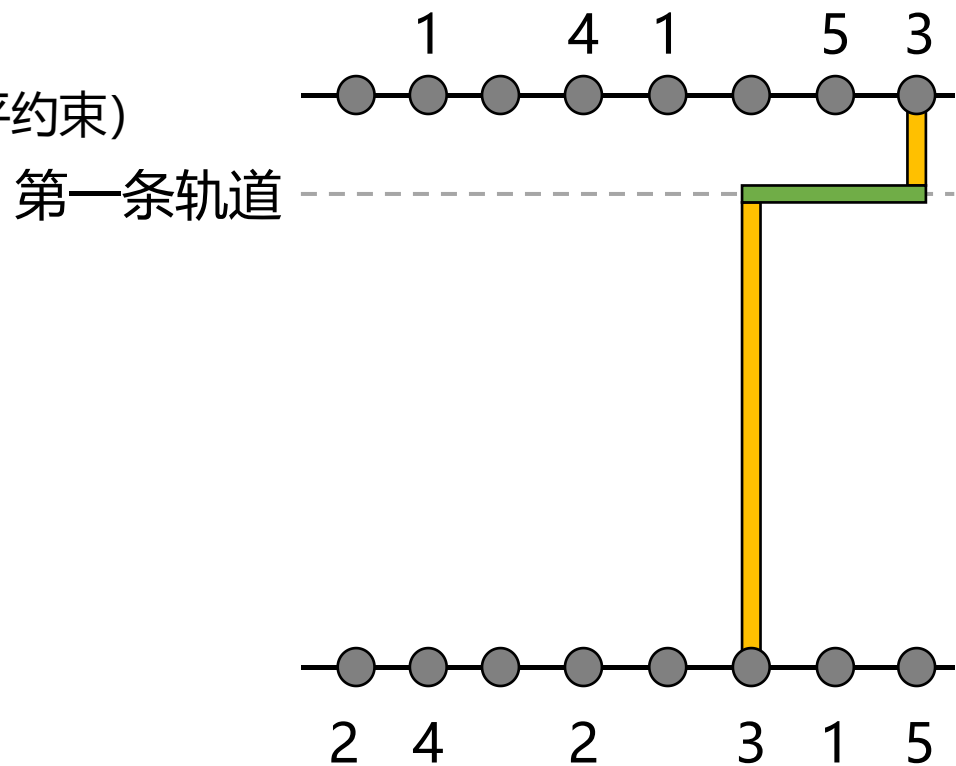
- 然后，对没有垂直约束的连接（在垂直约束图中入度为0的节点）按照顺序逐一布线。
如下所示，节点3的入度为0，所以3分配到第一条轨道上，并且布线。接着在垂直约束图中删除节点3及其相关边

(因为只选择了一个节点，独享整条轨道，所以不用检查是否有水平约束)

连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]



垂直约束图



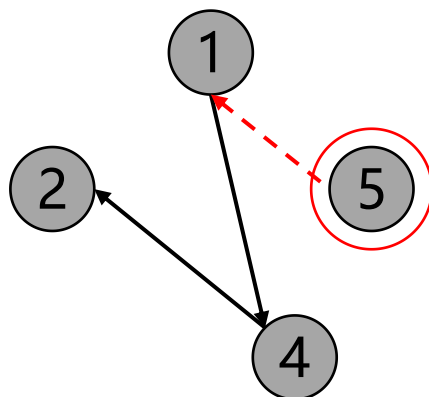
■ 详细布线算法

□ 左边算法例子

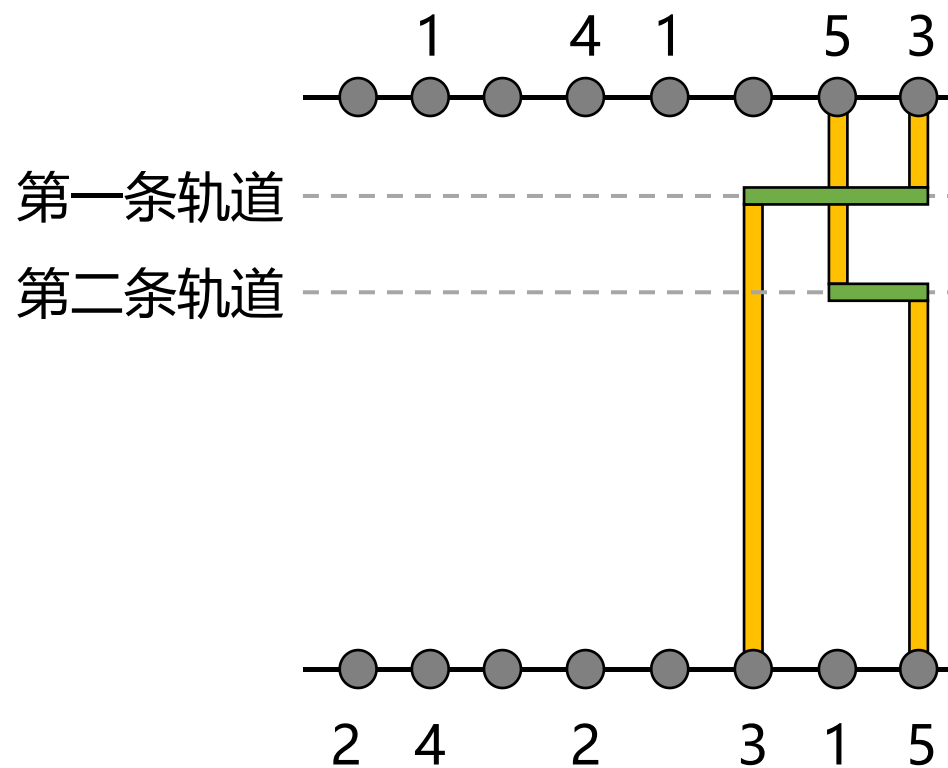
– 布线

- 在更新后的垂直约束图中，节点5的入度为0，所以5分配到第二条轨道上，并且布线。接着在垂直约束图中删除节点5及其相关边

连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]



垂直约束图



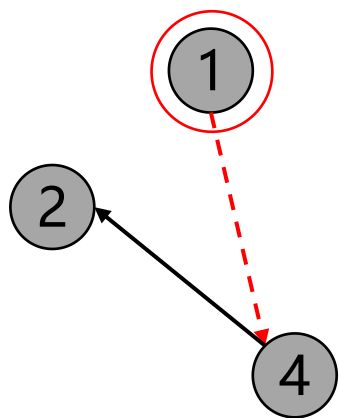
■ 详细布线算法

□ 左边算法例子

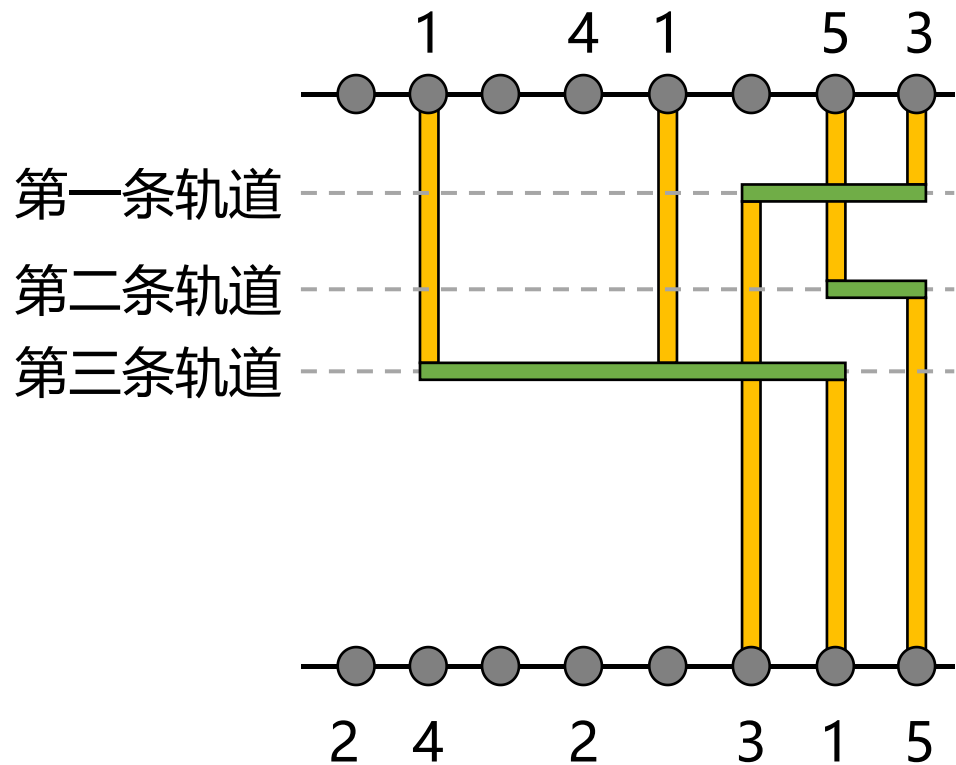
– 布线

- 在更新后的垂直约束图中，节点1的入度为0，所以1分配到第三条轨道上，并且布线。接着在垂直约束图中删除节点1及其相关边

连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]



垂直约束图



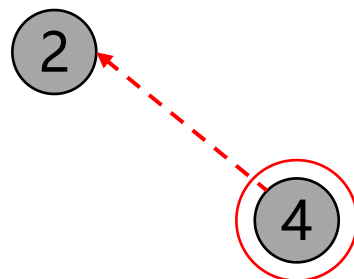
■ 详细布线算法

□ 左边算法例子

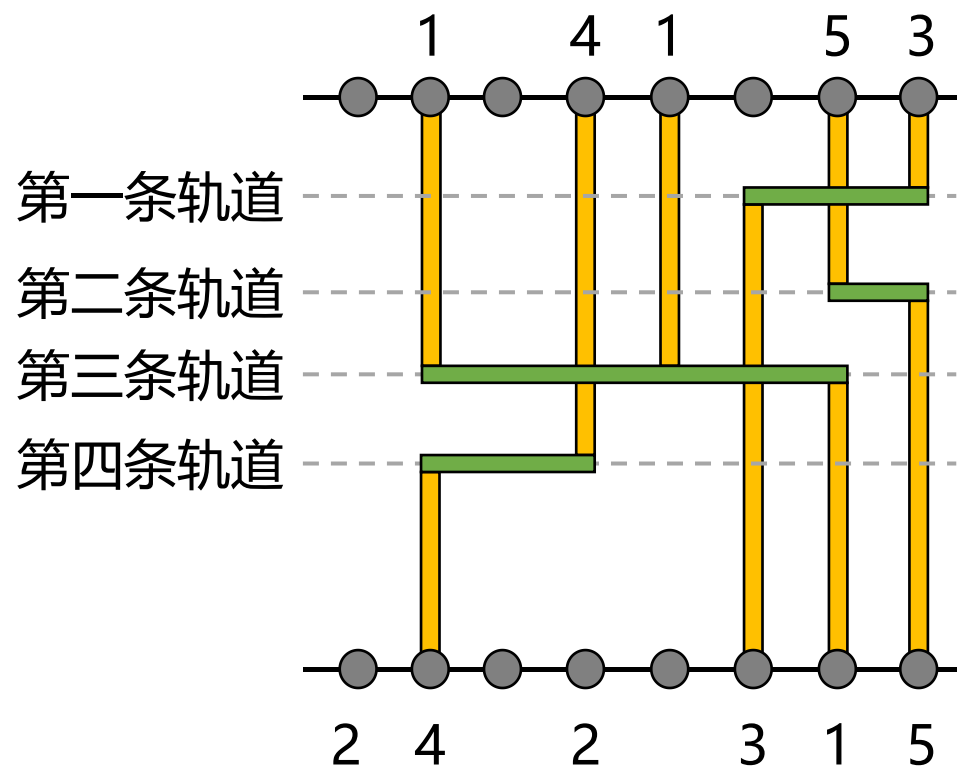
– 布线

- 如下所示，节点4的入度为0，所以4分配到第四条轨道上，并且布线。接着在垂直约束图中删除节点4及其相关边

连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]



垂直约束图



■ 详细布线算法

□ 左边算法例子

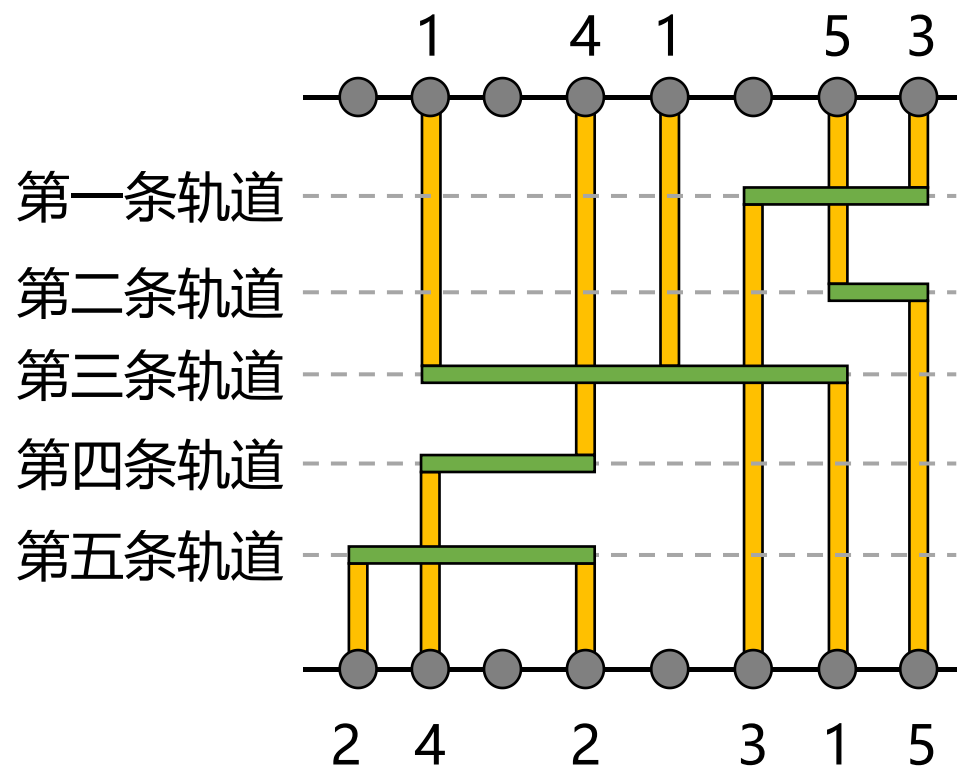
– 布线

- 如下所示，最后剩下节点2，所以2分配到第五条轨道上，并且布线。接下来垂直约束图没有节点，布线结束。布线高度为5

连接	范围
2	[1,4]
1	[2,7]
4	[2,4]
3	[6,8]
5	[7,8]

2

垂直约束图



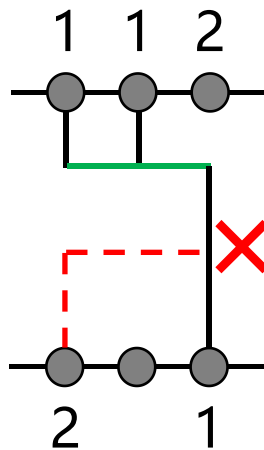
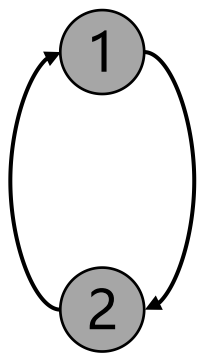
■ 详细布线算法

□ 左边算法总结

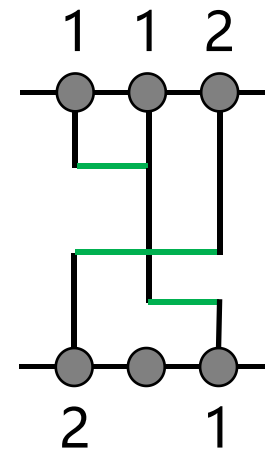
– 缺点

- 整个线网都位于一个轨道上面
- 不能处理垂直约束图上的约束循环

若连接1和2存在一个约束循环



在实际的布线上会导致冲突



狗腿布线算法分解多引脚线网可以处理约束循环

■ 详细布线算法

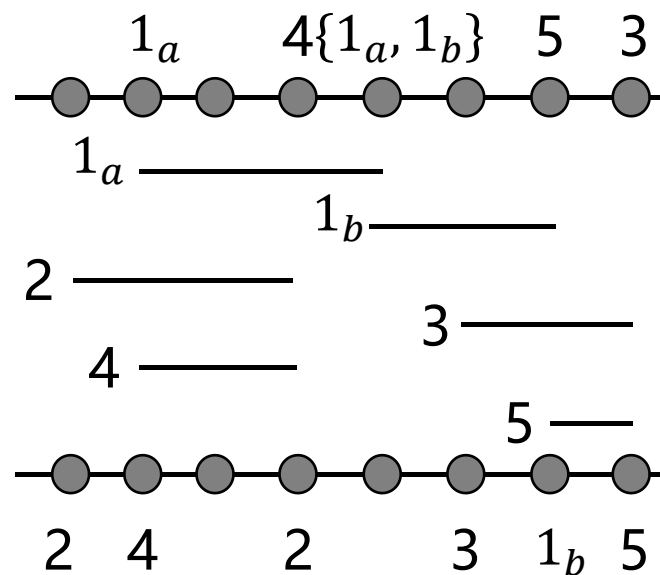
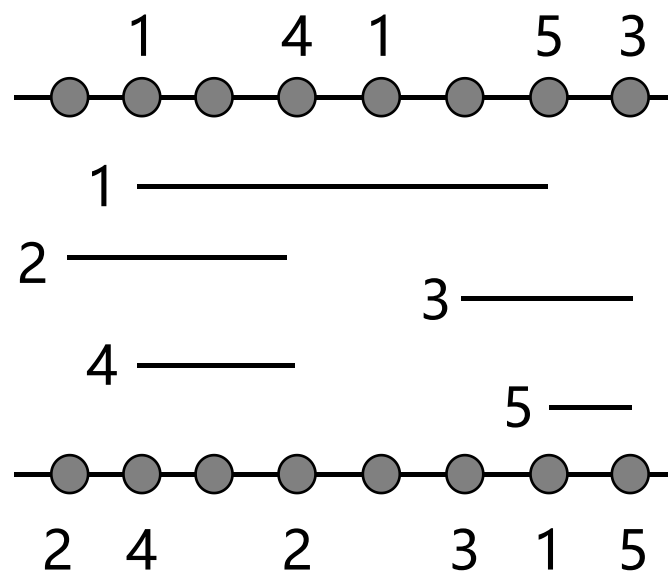
- 狗腿通道布线算法 (Dogleg channel routing)
 - 包含三个步骤
 - 分解：将每个多引脚线网分解为两引脚连接
 - 构建约束图：根据这些连接的位置，构建水平约束图和垂直约束图来建模布线约束
 - 布线：根据水平约束图和垂直约束图的约束条件布线每个线网

■ 详细布线算法

□ 狗腿通道布线算法例子

– 分解

➤ 由于1是多引脚线网，所以将1分解为 1_a 和 1_b

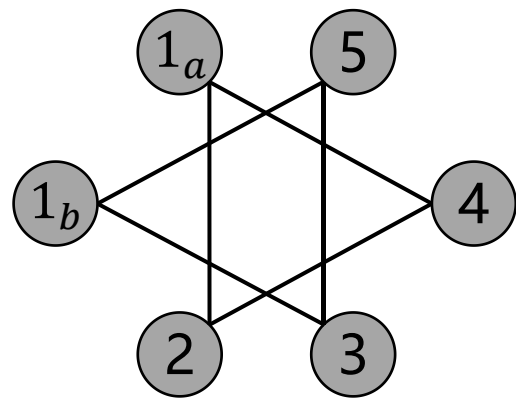
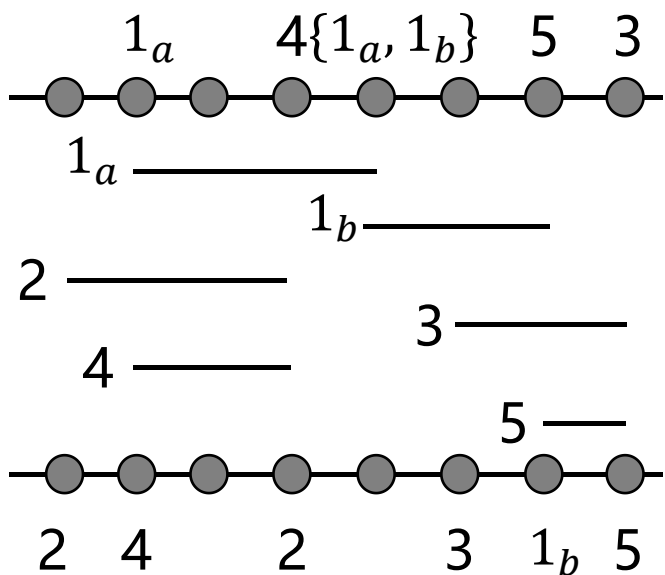


■ 详细布线算法

□ 狗腿通道布线算法例子

– 构建约束图

➤ 按照规则构建好水平约束图，因为 1_a 和 1_b 属于同一个线网，所以它们之间没有水平约束

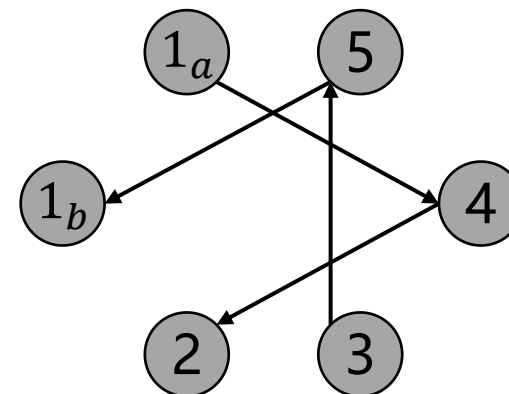
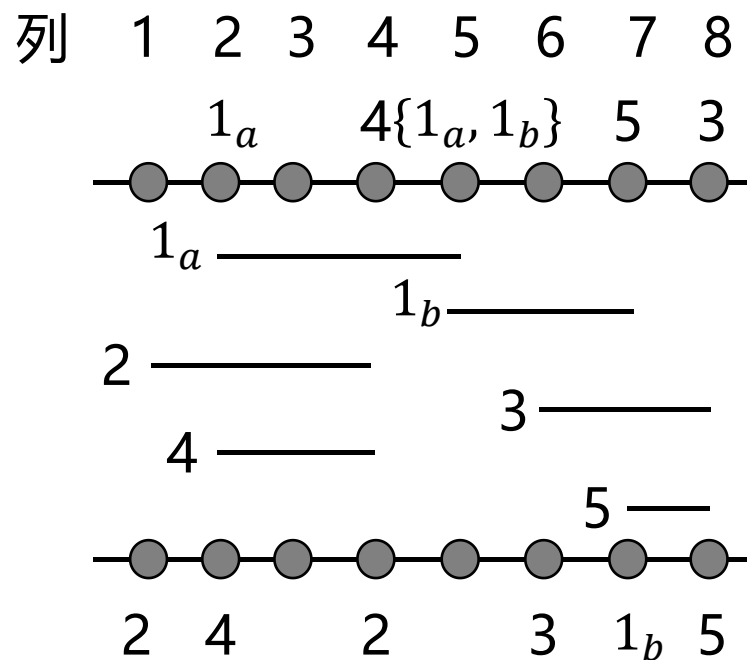


■ 详细布线算法

□ 狗腿通道布线算法例子

– 构建约束图

➤ 按照规则构造好垂直约束图

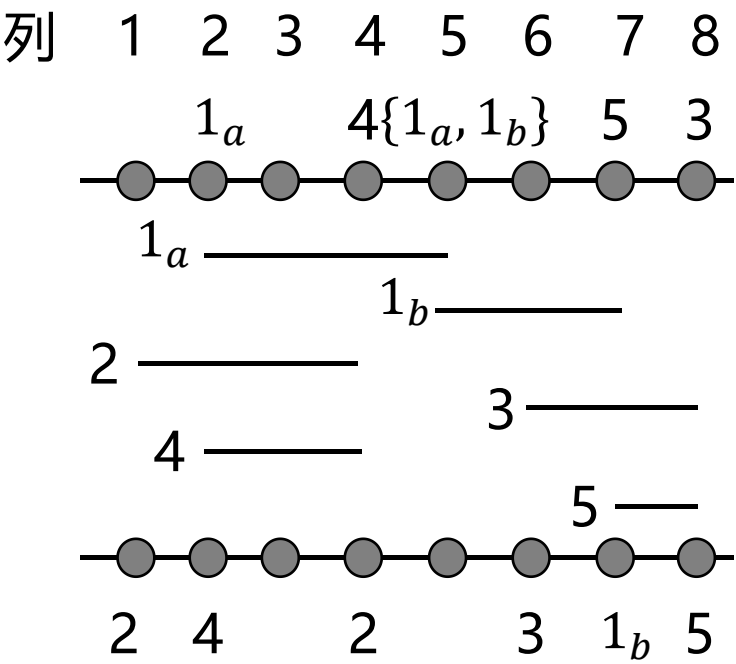


■ 详细布线算法

□ 狗腿通道布线算法例子

– 布线

➤ 首先，将每个连接进行排序，排序方式是根据连接区间的左端坐标进行升序排序



连接	范围
2	[1,4]
1 _a	[2,5]
4	[2,4]
1 _b	[5,7]
3	[6,8]
5	[7,8]

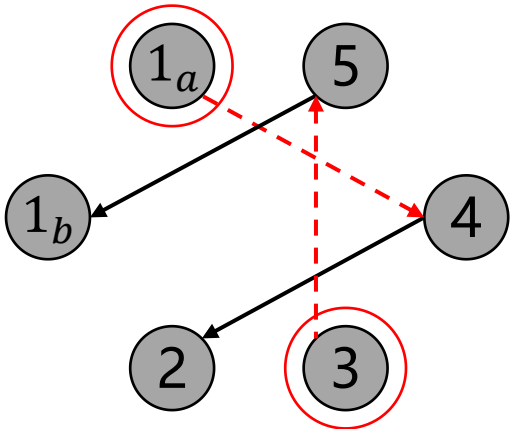
■ 详细布线算法

□ 狗腿通道布线算法例子

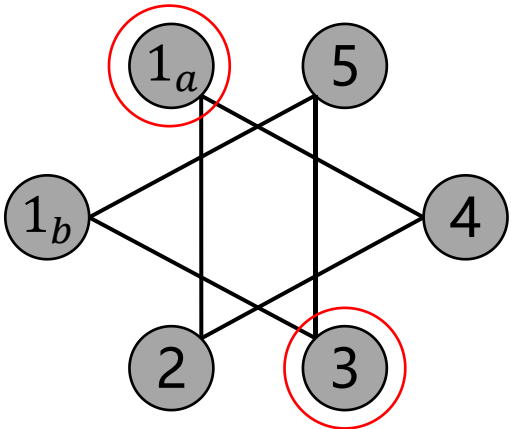
– 布线

- 然后，对没有垂直约束的连接（在垂直约束图中入度为0的节点）按照顺序逐一布线。
如下，节点 1_a 和3的入度为0，在水平约束图中 1_a 和3不存在水平约束，所以 1_a 和3可以分配到第一条轨道上，并且逐一布线。接着在垂直约束图中删除节点 1_a 和3及其相关边

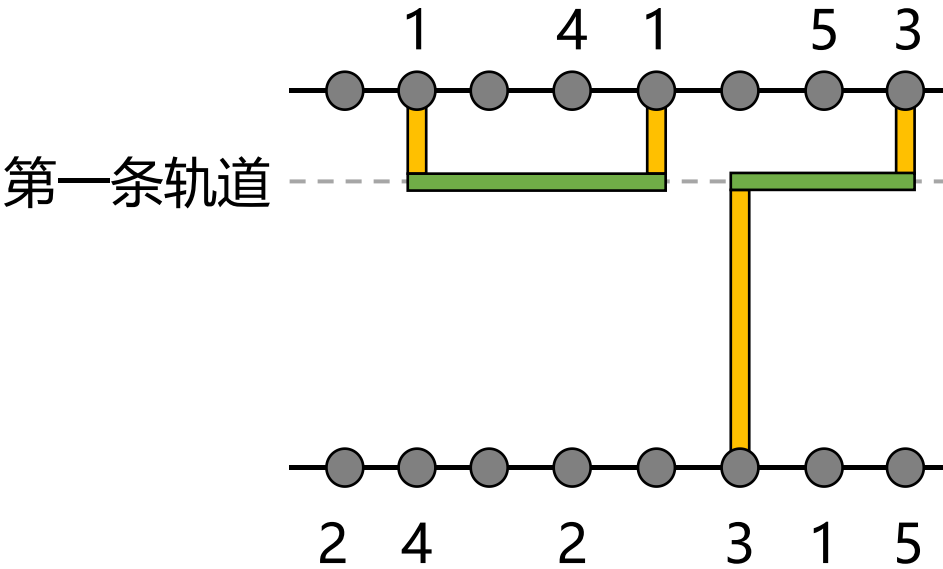
连接	范围
2	[1,4]
1_a	[2,5]
4	[2,4]
1_b	[5,7]
3	[6,8]
5	[7,8]



垂直约束图



水平约束图



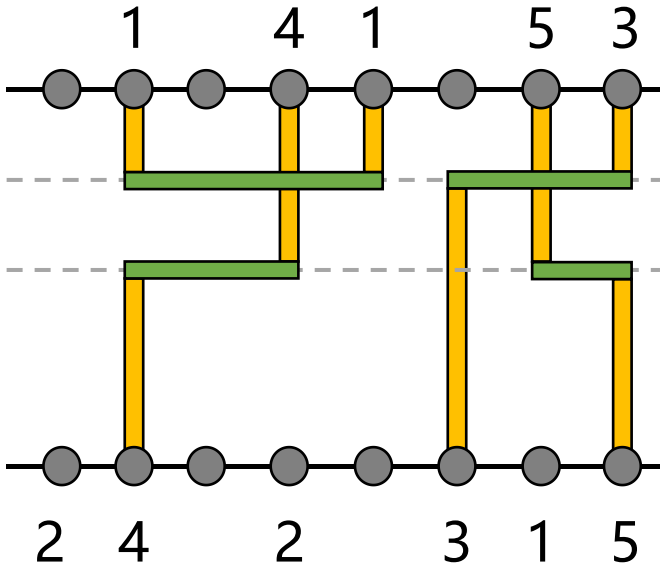
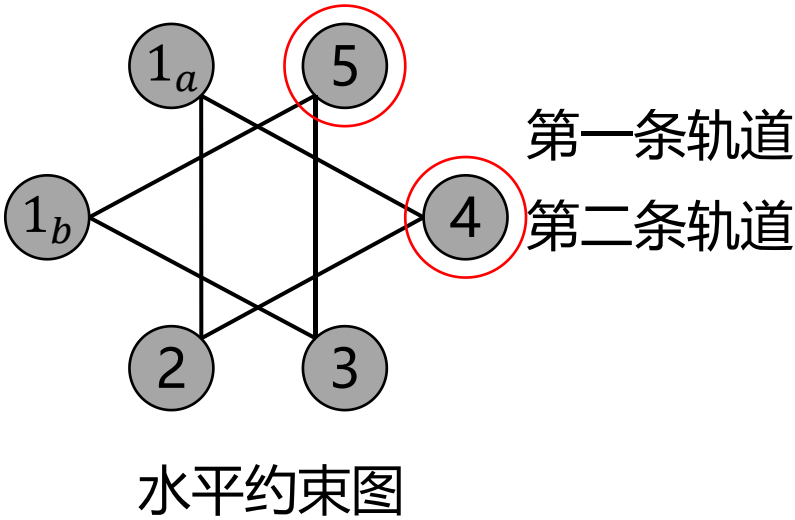
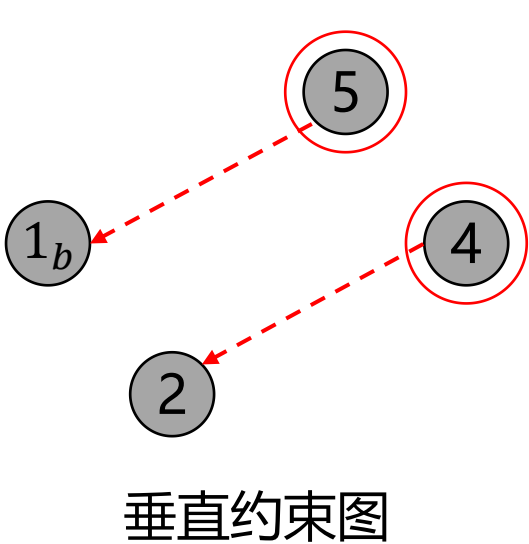
详细布线算法

狗腿通道布线算法例子

– 布线

- 在更新后的垂直约束图中，节点4和5的入度为0，在水平约束图中4和5不存在水平约束，所以4和5可以分配到第二条轨道上，并且逐一布线。接着在垂直约束图中删除节点4和5及其相关边

连接	范围
2	[1,4]
1 _a	[2,5]
4	[2,4]
1 _b	[5,7]
3	[6,8]
5	[7,8]



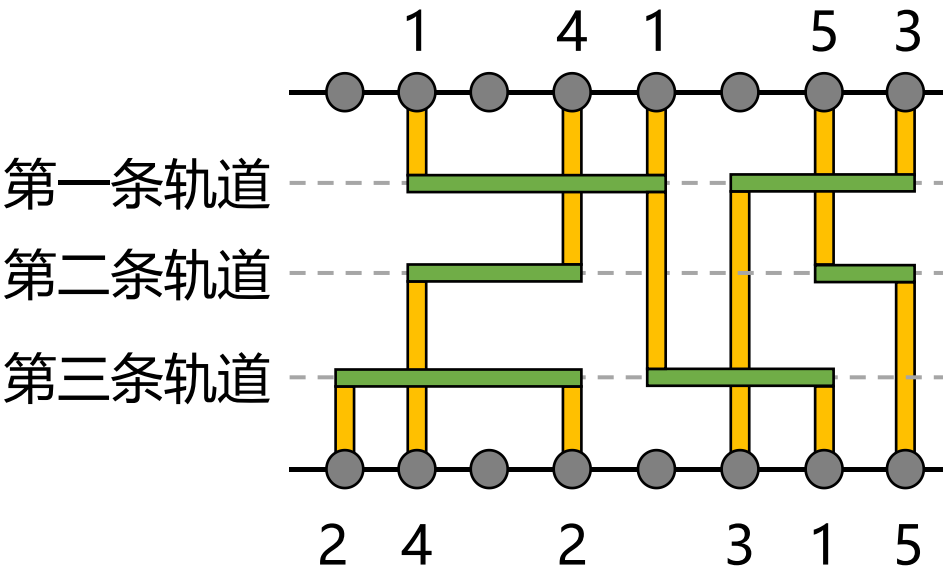
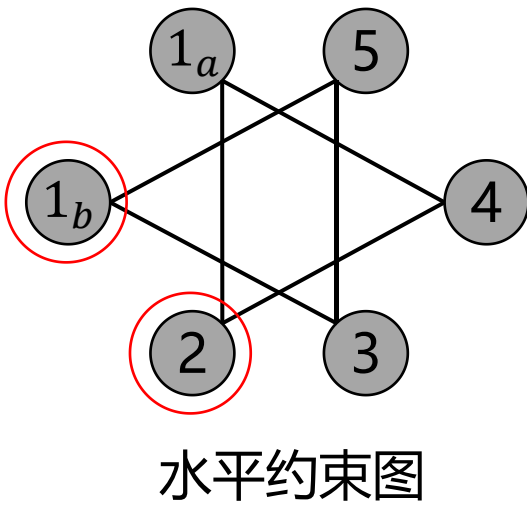
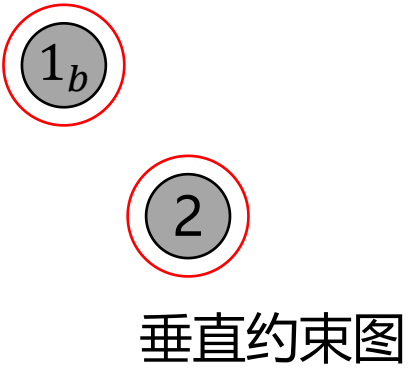
■ 详细布线算法

□ 狗腿通道布线算法例子

– 布线

➤ 在更新后的垂直约束图中，节点 1_b 和2的入度为0，在水平约束图中 1_b 和2不存在水平约束，所以 1_b 和2可以分配到第三条轨道上，并且逐一布线。接着在垂直约束图中删除节点 1_b 和2及其相关边。接下来垂直约束图没有节点，布线结束。布线高度为3

连接	范围
2	[1,4]
1_a	[2,5]
4	[2,4]
1_b	[5,7]
3	[6,8]
5	[7,8]



■ 详细布线算法

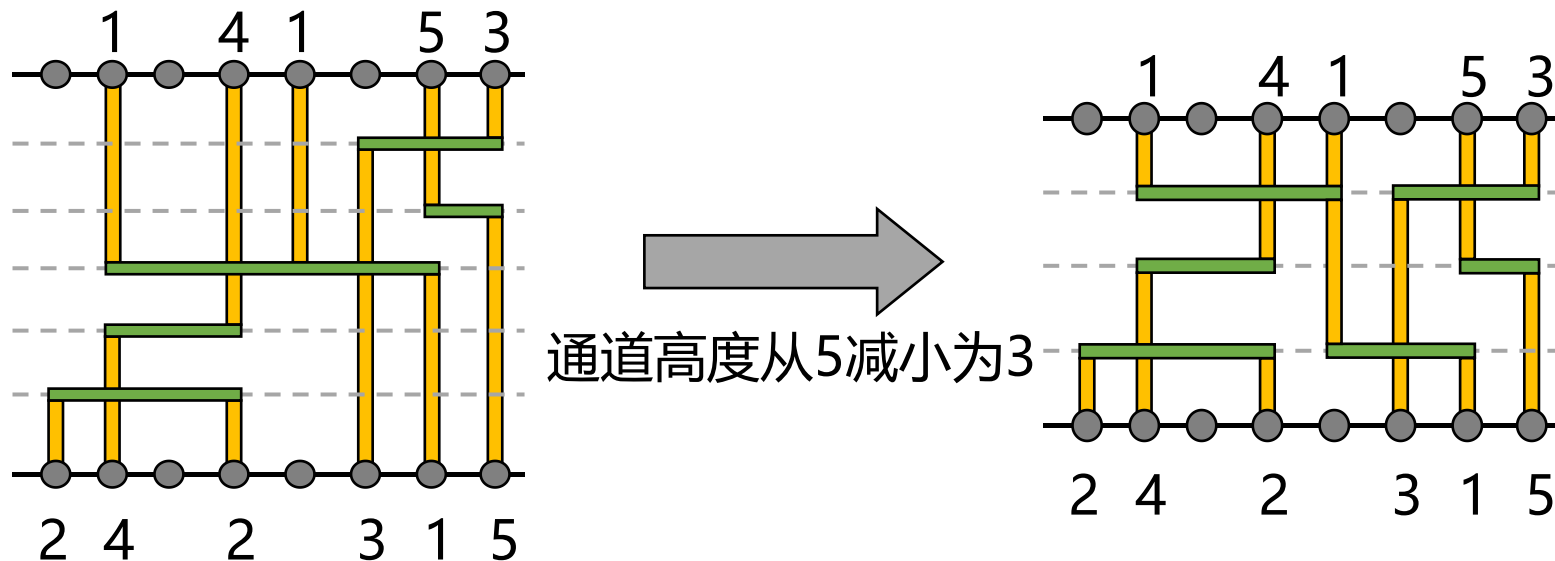
□ 狗腿通道布线算法总结

– 优点

- 对比左边算法，优化通道高度
- 可以处理约束循环

– 缺点

- 将线网的部分放置在不同的轨道上，但引入了额外的通孔，导致成本增加



感谢！
